Modelos de Lenguaje

Resumen para el Primer Parcial — Unidades 1 y 2

UTN Facultad Regional Mendoza

Armado a partir de los cinco Colabs de cátedra (Unidad 1 — Código; Introducción a Embeddings; Unidad 2 — Clase 1: RNN; Clase 2: Modelado de lenguaje; Clases 3 y 4: Traducción y Atención) y de los bancos de autoevaluación de ambas unidades, con sus correcciones. Cada tema incluye la definición, las fórmulas y formas de tensores, los números de referencia del notebook, la lectura de los gráficos importantes y los puntos donde el banco de preguntas suele tender trampas.

Índice

UNIDAD 1 — Del texto al clasificador

1. Tokenización y vocabulario
2. Representación en tensores: relleno y máscara
3. Embeddings como capa lineal
4. El perceptrón y la capa lineal vectorizada
5. Funciones de activación y no linealidad
6. Arquitectura del clasificador de reseñas
7. Capa de salida según la tarea
8. Pérdidas: L1, L2 y entropía cruzada
9. Autograd, gradiente y descenso
10. El loop de entrenamiento completo
11. Dataset, DataLoader y collate_fn
12. Esquemas de actualización (full-batch, incremental, mini-batch)
13. Optimizadores (SGD, momentum, RMSProp, Adam)
14. Entrenamiento, validación y prueba
15. Overfitting, underfitting y regularización
16. Métricas: accuracy, precisión, recall, matriz de confusión
17. Guardar y cargar modelos
18. Embeddings aprendidos: qué queda al final

UNIDAD 2 — Secuencias, memoria y atención

19. Hipótesis distribucional
20. Ventanas, co-ocurrencia y skip-gram
21. Muestreo negativo
22. word2vec, GloVe y fastText
23. Redes recurrentes (RNN): forma a mano y costo
24. Arquitecturas según la tarea (uno-a-muchos, muchos-a-uno, etc.)
25. Relleno en secuencias: pack_padded_sequence
26. Desvanecimiento del gradiente: LSTM y GRU
27. Modelado de lenguaje y autosupervisión
28. Perplejidad
29. Generación de texto: temperatura, top-k, top-p
30. Traducción: encoder-decoder
31. Teacher forcing y sesgo de exposición
32. BLEU
33. Beam search
34. El cuello de botella del vector único
35. Atención de Bahdanau
36. Consulta, clave y valor (QKV)
37. Atención multi-cabeza
38. Los dos tipos de "pesos" en atención (nota de clase)

UNIDAD 1 — Del texto al clasificador

Recorrido de la unidad: texto → tokens → índices → tensor `(B, L)` → embedding → promedio → capas lineales con activación → logits → pérdida → gradiente → actualización → métricas → guardar el modelo. El caso de uso de referencia es un clasificador de reseñas en 5 clases (1 a 5 estrellas).

1. Tokenización y vocabulario

Qué hace un tokenizador

El tokenizador es la **primera puerta**: decide qué información puede llegar al modelo. Lo que se descarta ahí desaparece para siempre; el modelo nunca lo ve, y no hay forma de que lo "recupere después" por contexto.

> Ejemplo del notebook: el tokenizador simple se queda con secuencias alfanuméricas y pasa todo a minúscula. Con «Muy mala :(PÉSIMO!!! $12.500 tirados», el «:(» y los «!!!» desaparecen, y «$12.500» se parte en «12» y «500». Esa pérdida es de **información**, no solo de vocabulario ni de cómputo.

Costos medidos: conservar puntuación agrega una decena de miles de tokens de vocabulario y ~11% de largo extra en las secuencias. Conservar mayúsculas agranda el vocabulario ~20% (23.067 → 27.553 tokens) y **empeora** las métricas finales (más parámetros por aprender, menos ejemplos por palabra). Quitar acentos achica el vocabulario ~6% pero pierde distinciones reales (papa/papá).

Vocabulario: cobertura vs. tasa de <unk>

El vocabulario se arma con las palabras del corpus de **entrenamiento** que aparecen al menos `freq_min` veces; el resto se reemplaza por `<unk>`.

Medida	Qué mide	Sobre qué corpus
Cobertura	Fracción de apariciones que quedan dentro del vocabulario	El corpus de **entrenamiento** (con el que se construyó)
Tasa de <unk>	Fracción de tokens de un texto cualquiera que caen en <unk>	Cualquier texto nuevo (test)

Son medidas distintas y **pueden diferir**: la cobertura no ve nunca el test, y el test puede tener palabras que jamás aparecieron en entrenamiento. Con `freq_min=1` la cobertura es 100% pero la tasa de <unk> en test no baja de un piso irreducible (0,23% en el notebook): son palabras que simplemente nunca estuvieron en entrenamiento.

Efecto de `freq_min`: subir de 1 a 50 achica el vocabulario ~10 veces (59.045 → 5.882 tokens) por solo ~5 puntos menos de cobertura. Esto es consecuencia de **Zipf**: pocas palabras muy frecuentes cubren casi todo el texto, y muchísimas palabras raras aportan poca cobertura cada una.

2. Representación en tensores: relleno y máscara

Truncar y rellenar

Las reseñas tienen largo variable; para armar un tensor rectangular `(B, L)` se fija un largo `L`: las reseñas más largas se **truncan** y las más cortas se **rellenan** con `<pad>` (índice 0) hasta llegar a `L`.

Percentiles de largo del corpus (200.000 reseñas): mediana 25-26, p75 = 38, p90 = 59, p95 = 76, p99 = 123-127. Con `L = 60`: aproximadamente el **10%** de las reseñas se trunca (10% × 200.000 = **20.000**), porque el percentil 90 es 59. Subir `L` a 123 (p99) bajaría el truncado al 1% pero duplicaría el cómputo; bajarlo a la mediana truncaría a la mitad de las reseñas.

Cálculo genérico: % de reseñas que se truncan con un L dado ≈ el percentil cuyo valor es igual a L. Si el largo promedio es 32 y L = 60, el **porcentaje de relleno** en el tensor (no de truncado) es (60 − 32) / 60 ≈ **47%** (medido en el notebook da un poco más de la mitad, porque la distribución es asimétrica — la mediana de 26 y las reseñas truncadas a 60 empujan el número hacia arriba).

Forma y dtype del tensor

Lote de B reseñas con largo fijo L → tensor de forma (B, L), dtype torch.int64 (long)

Es `int64` y no `float32` porque son **índices** de una tabla, no cantidades. El índice 42 no es "más grande" que el índice 7; son casilleros.

La máscara de relleno

máscara = (lote != 0) → forma (B, L), la misma que el lote; dtype bool

Vale `True` en las posiciones con un **token real** y `False` en el relleno. Se usa después para promediar los embeddings sin contar el relleno (promedio enmascarado).

¿Por qué NO se normalizan los índices?

La teoría general dice que las entradas de una red se normalizan (media 0, desvío 1, o rango 0–1). Acá **no se aplica**: los índices de vocabulario son **casilleros de una tabla**, no cantidades comparables. Normalizarlos (por ejemplo dividiendo por el tamaño del vocabulario) sería tan absurdo como promediar números de documento. El rol que cumple la normalización en este problema lo cumple la **capa de embedding**: convierte cada índice en un vector de floats de escala razonable y aprendida. (En arquitecturas más grandes la normalización reaparece, pero sobre activaciones internas — *layer normalization* —, no sobre los índices de entrada.)

Padding dinámico y su límite

Idea: en vez de rellenar todos los lotes hasta L=60, cada lote de 256 reseñas al azar se rellena solo hasta el largo de su reseña **más larga**. El ahorro medido fue **cero**.

Por qué no ahorra: con el 10% de las reseñas llegando al tope de 60, en un lote de 256 tomadas al azar casi siempre hay al menos una reseña larga, y con una sola alcanza para que todo el lote se rellene

> hasta 60 igual. La solución real es **bucketing**: agrupar reseñas de largo parecido antes de armar los lotes (usando `batch_sampler`), lo que sí ahorró casi la mitad del cómputo — a costa de que los lotes dejan de ser muestras puramente aleatorias.

---

3. Embeddings como capa lineal

Qué es `nn.Embedding`

> `nn.Embedding(V, E)` es una **capa lineal sin sesgo**, con entrada one-hot, implementada como **búsqueda en una tabla** (V filas, una por palabra; E columnas, la dimensión del vector). Buscar la fila *i* es matemáticamente lo mismo que multiplicar el one-hot de *i* por la tabla completa; la búsqueda solo evita hacer ese producto absurdo con un vector casi todo ceros.

Consecuencias importantes: es un **parámetro más** de la red (se entrena con el resto, no es preprocesamiento fijo), y es la **primera capa** de cualquier modelo de lenguaje o clasificador de texto.

$$\text{Entrada } (B, L) \text{ de índices} \rightarrow \text{nn.Embedding}(V, E) \rightarrow \text{salida } (B, L, E)$$

Promedio enmascarado

Para pasar de "un vector por palabra" a "un vector por reseña" se promedia sobre la dimensión L, usando la máscara para no contar el relleno.

$$(B, L, E) + \text{máscara } (B, L) \rightarrow \text{promedio enmascarado} \rightarrow (B, E)$$

> **Ejemplo de banco:** clasificador con 10 categorías en vez de 5 estrellas, con `nn.Embedding(V, 64)`. Después del embedding: `(B, L, 64)`. Después del promedio con máscara: `(B, 64)`. A la salida de la última `nn.Linear`: `(B, 10)` — un logit por categoría (la trampa típica es poner (B, 5), que era la cantidad de clases del ejemplo de estrellas).

---

4. El perceptrón y la capa lineal vectorizada

Un perceptrón

Un perceptrón con *n* entradas calcula `z = w·x + b`: una combinación lineal de las entradas más un sesgo.

Parámetros de `nn.Linear(entradas, salidas)`

$$\text{parámetros} = (\text{entradas} \times \text{salidas}) + \text{salidas} \quad [\text{pesos} + \text{sesgos}]$$

`nn.Linear(2, 3)`: pesos 3×2 = 6, sesgos 3 → total **9** parámetros.
Referencia de la Unidad 2: una RNN a mano con 2 entradas y 3 unidades ocultas tiene **21** parámetros (W_ih 3×2=6, W_hh 3×3=9, b_ih 3, b_hh 3), contra 9 de esta capa lineal simple: los 12 "de más" son toda la memoria.

Cálculo vectorizado de una capa (todo el lote de una vez)

Con los ejemplos en filas, `X` de forma `(B, entradas)`, y la matriz de pesos `W` de forma `(salidas, entradas)` (convención de PyTorch: cada **fila** de W es un perceptrón):

$$Z = X \mathbin{@} W.T + b \;\rightarrow\; \text{forma } (B, \text{salidas})$$

La transposición hace coincidir las dimensiones internas: `(B, entradas) @ (entradas, salidas) = (B, salidas)`. El sesgo `b`, de forma `(salidas,)`, se estira por **broadcasting** y se suma a cada fila del lote.

> Trampas típicas: `W @ X + b` (orden de matrices incorrecto — no coinciden dimensiones), `X * W + b` (producto elemento a elemento, no es lo que hace una capa lineal), o un `for` explícito sobre las B filas (funciona pero es exactamente lo que la vectorización evita).

---

5. Funciones de activación y no linealidad

Las tres activaciones clásicas

Función	Rango	Nota
Sigmoide	(0, 1)	Salida binaria / probabilidad
Tanh	(−1, 1)	Centrada en 0
ReLU	[0, ∞)	0 si entrada < 0, identidad si ≥ 0. La más usada: barata y no lineal

Por qué hace falta una no linealidad entre capas

> **Apilar capas lineales sin ninguna activación entre ellas es exactamente lo mismo que una sola capa lineal.** Demostración algebraica: $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$, que tiene la forma de una única transformación lineal. Esto vale para cualquier cantidad de capas apiladas sin activación: toda la profundidad no compra nada. Con una ReLU en el medio, ya no existe ninguna matriz única que reproduzca el cálculo, y con suficientes unidades el MLP se vuelve un **aproximador universal** de funciones. Cada quiebre en la función de salida corresponde a una ReLU activándose o apagándose.

Comprobación gráfica del notebook: "3 capas sin activación" siempre da una **recta**, incluso con pesos al azar. "3 capas con ReLU" da una curva quebrada, también con pesos al azar y sin entrenar.

Capa de salida según la tarea

Tarea	Capa de salida	Ejemplo
Regresión	**1 unidad, sin activación**	Precio de una casa
Clasificación binaria	**1 unidad + sigmoide**	¿Hay un perro en la foto?
Clasificación multiclase	**N unidades + softmax**	Reseña de 1 a 5 estrellas → 5 unidades

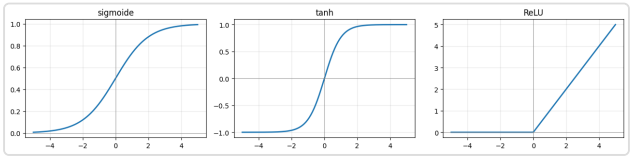
> "2 unidades + sigmoide" es casi siempre la opción de más / trampa en los emparejamientos: dos sigmoides independientes no forman una distribución de probabilidad (no suman 1 necesariamente). Serviría para **multietiqueta** (cuando una entrada puede pertenecer a varias clases a la vez), que no es ninguno de estos casos.

---

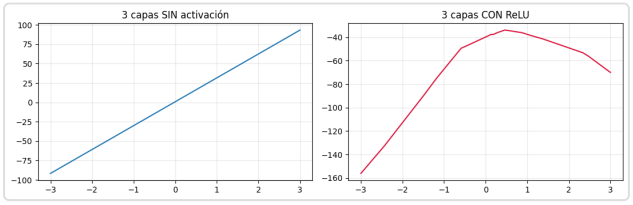
6. Arquitectura del clasificador de reseñas

índices (B, L) → Embedding(23.067, 64) → (B, L, 64) → promedio enmascarado → (B, 64) → Linear(64, 128) + ReLU → (B, 128) → Linear(128, 5) → (B, 5) logits

Total: 1.485.253 parámetros, de los cuales **1.476.288** (99,4%) están en la tabla de embeddings. Solo 8,3 parámetros del modelo "le tocan" a cada ejemplo de entrenamiento. Esta concentración en la tabla es la raíz del sobreajuste que se ve más adelante en la unidad.



Sigmoide, tanh y ReLU



3 capas sin activación (recta) vs. con ReLU (curva quebrada)

7. Pérdidas: L1, L2 y entropía cruzada

Pérdidas de regresión: L1 y L2

Pérdida	Fórmula	Cuándo usarla
L1 (error absoluto medio, MAE)	promedio(|predicho − real|)	Hay outliers que se consideran **ruido** y no deben dominar el ajuste (precio de casas con valores disparatados)
L2 (error cuadrático medio, MSE)	promedio((predicho − real)²)	Un error grande en un solo caso es **mucho más grave** que varios chicos (tiempo de entrega)

Ejemplo de cálculo: predicho = [2,5; 0,0; 2,0; 8,0], real = [3,0; −0,5; 2,0; 7,0].
Errores absolutos: 0,5 / 0,5 / 0 / 1 → L1 = (0,5+0,5+0+1)/4 = **0,500**.
Errores al cuadrado: 0,25 / 0,25 / 0 / 1 → L2 (MSE) = (0,25+0,25+0+1)/4 = **0,375**.
Con un atípico (por ejemplo un real = 17 en el último caso), L1 sube ~5 veces y L2 más de 100 veces: elevar al cuadrado amplifica mucho los errores grandes.

Logits, softmax y entropía cruzada

La última capa de un clasificador produce **logits**: números crudos, pueden ser negativos, no suman 1 (ejemplo: [1,4; −2,1; 3,5]). Para leerlos como probabilidades hay que pasarlos por la **softmax**: $\text{softmax}(z)_i = \exp(z_i) / \Sigma_j \exp(z_j)$, que exponencia y normaliza.

nn.CrossEntropyLoss recibe: logits crudos como predicción (forma (N, clases)) y el índice de la clase como etiqueta (forma (N,))

Por dentro aplica log-softmax y toma el valor de la clase correcta con signo cambiado — es la entropía cruzada contra el one-hot, calculada de forma más estable y sin construir el tensor de ceros.

El bug silencioso más importante de la materia: si el modelo ya aplica `softmax` en su `forward` y además se entrena con `nn.CrossEntropyLoss`, la pérdida aplica su propia softmax sobre números que ya están entre 0 y 1. El resultado es casi uniforme, el gradiente se achica, y el modelo **entrena pero mucho peor** — sin ningún error que lo delate. Por convención, el modelo siempre devuelve logits crudos.

Entropía cruzada de un ejemplo = −ln(p_correcta). Con p = 0,9 → −ln(0,9) = **0,105**. Con p = 0,5 → 0,69. Con p = 0,1 → 2,30. Cuanta más probabilidad se le da a la clase correcta, más baja la pérdida; equivocarse con seguridad sale carísimo.

Verificación de que el modelo arranca bien: sin entrenar, con N clases balanceadas y pesos al azar, la salida es aproximadamente uniforme → p(correcta) ≈ 1/N → pérdida inicial ≈ **ln(N)**. Con 5 clases: ln(5) ≈ **1,609** (el notebook mide 1,60–1,61 según el lote). Si la pérdida inicial no arranca cerca de ln(nº de clases), hay un bug en la inicialización, las etiquetas o las formas de los tensores.

8. Autograd, gradiente y descenso

Qué es el gradiente y qué hace el paso de descenso

> Si `w.grad = +3,2` después de `backward()`: la derivada dice que **aumentar** ese peso haría **subir** la pérdida. El paso de descenso lo mueve en **sentido contrario** al gradiente: `p -= lr * p.grad`. No se resta el gradiente entero de una vez (eso no llevaría al mínimo); el tamaño real del paso lo regula la tasa de aprendizaje.

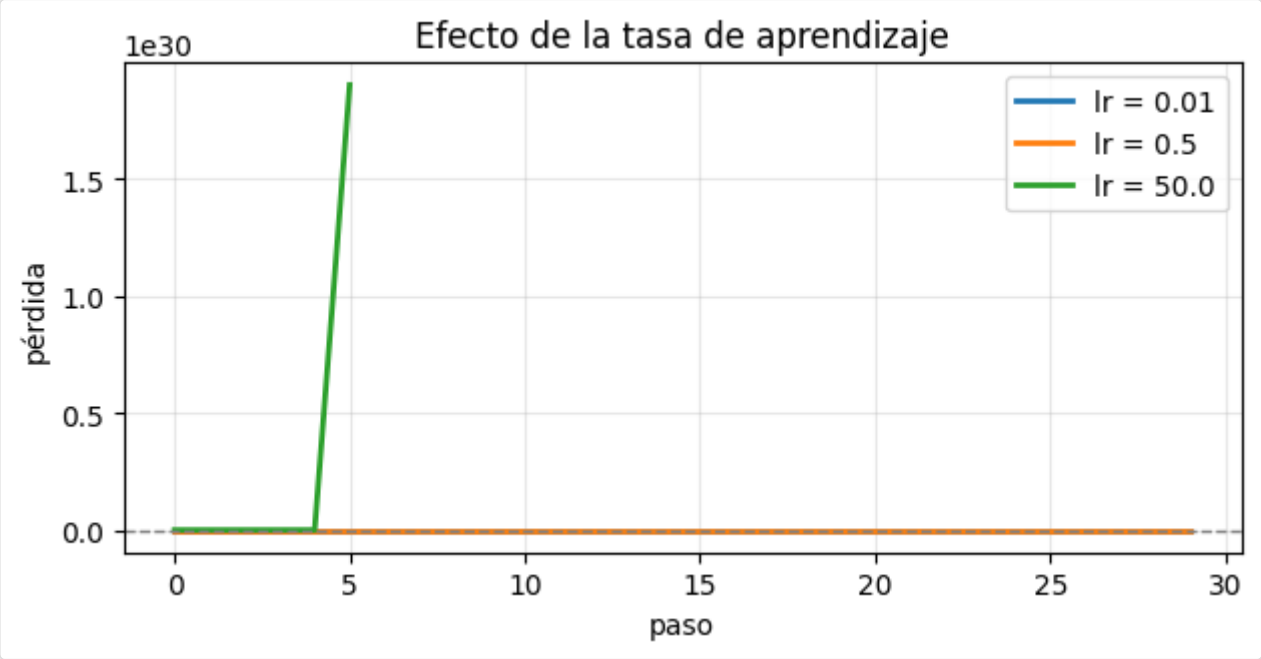
Por qué no probar pesos al azar en vez de usar el gradiente: el espacio de combinaciones de una red real (millones de parámetros) es tan enorme que acertar por azar tiene probabilidad prácticamente nula. El gradiente da una **dirección local** hacia donde bajar la pérdida; ese es todo el algoritmo, repetido muchas veces.

Acumulación de gradientes

> `backward()` **suma** al `.grad` existente, no lo reemplaza. Dos llamadas seguidas de `backward()` sobre el mismo lote sin limpiar nada, con derivada 12 cada vez, dejan `w.grad = 24`. Lo que evita que cada paso use la suma de toda la historia del entrenamiento es `optimizador.zero_grad()`, que se llama **antes de `backward()`, en cada iteración**. Olvidarse de esto es el bug más frecuente y silencioso del deep learning: no tira error, solo entrena mal.

Tasa de aprendizaje (learning rate)

> Con lr muy chica (0,01): el modelo casi no se mueve, converge muy lento. Con lr razonable (0,5): baja rápido y se estabiliza. Con lr muy grande (50): el paso es tan largo que **se pasa del mínimo en cada iteración** y termina más arriba de donde empezó — **diverge**. En el gráfico del notebook, con lr=50 la pérdida llega a ~10^{30} en 30 pasos.

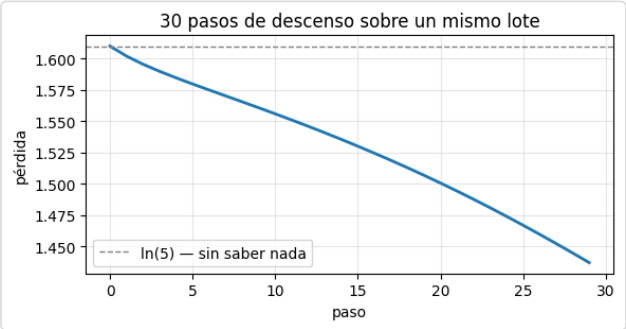


Efecto de la tasa de aprendizaje: 0,01 / 0,5 / 50

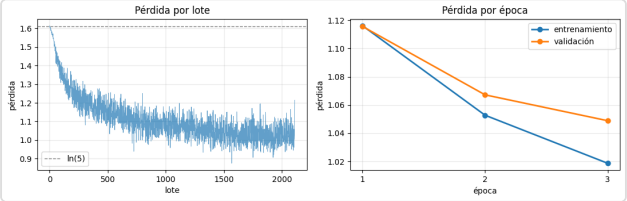
9. El loop de entrenamiento completo

Paso 1: `zero_grad()` — limpiar gradientes acumulados
Paso 2: forward — el lote atraviesa el modelo y salen los logits
Paso 3: calcular la pérdida, comparando logits con etiquetas
Paso 4: backward — recorrer el grafo hacia atrás y obtener el gradiente
Paso 5: `step()` — actualizar cada parámetro en sentido opuesto a su derivada

Este orden es fijo: **no hay gradiente antes de la pérdida, y no hay pérdida antes del forward.** `zero_grad()` puede ir al principio del loop o justo antes de `backward()`; lo único que no puede cambiar es que preceda al `backward()` de esa iteración.



30 pasos memorizando un único lote (arranca en ln 5)



Pérdida por lote (ruidosa) y por época (suave)

10. Dataset, DataLoader y collate_fn

Objeto	Responsabilidad
Dataset	Implementa `__len__` y `__getitem__`. Devuelve **un ejemplo por vez**, como tupla (entrada, etiqueta).
DataLoader	Agrupa en **lotes** y **baraja** entre épocas (si `shuffle=True`).
collate_fn	Convierte una lista de tuplas sueltas (ej. 256) en **dos tensores apilados** (por defecto, con `torch.stack`).
batch_sampler	Elige qué índices van juntos en cada lote; es lo que permite el *bucketing*.

Por qué `shuffle=True` no es un detalle cosmético

> El corpus de reseñas viene **ordenado por etiqueta** (primero todas las de 1 estrella, después las de 2, etc.). Barajar hace **estocástico** el descenso por mini-lotes: la red no ve siempre las mismas secuencias de ejemplos. Sin barajar, cada época mostraría las cinco clases en bloque, una detrás de otra: las actualizaciones desarrollarían patrones que se cancelan y el entrenamiento se degrada. "Estocástico" en SGD alude justamente a esta selección aleatoria. Validación y prueba **no** se barajan: el orden no afecta el resultado y conviene la reproducibilidad.

> **Split por posición sobre un corpus ordenado por clase:** si se toma el primer 90% para entrenar y el último 10% para validar (sin barajar antes), la validación queda con **una sola clase** (la última, ej. 5 estrellas). Un modelo que prediga siempre "5" daría 100% de accuracy en esa validación, sin significar nada. Por eso `random_split` baraja antes de partir.

---

11. Esquemas de actualización

Esquema	Descripción
Full-batch	Una sola actualización por época, calculada sobre el conjunto **entero**. Camino estable pero costoso; a veces el conjunto no entra en memoria.
Incremental (SGD puro)	Una actualización por **ejemplo**. La red se mueve muchísimo; cada paso persigue el error de un único caso y el recorrido es errático.
Mini-batch	Gradiente calculado sobre 32, 64 o 128 ejemplos. Menos ruidoso que de a uno, eficiente, aprovecha el paralelismo de la GPU. Es el equilibrio que se usa en la práctica.

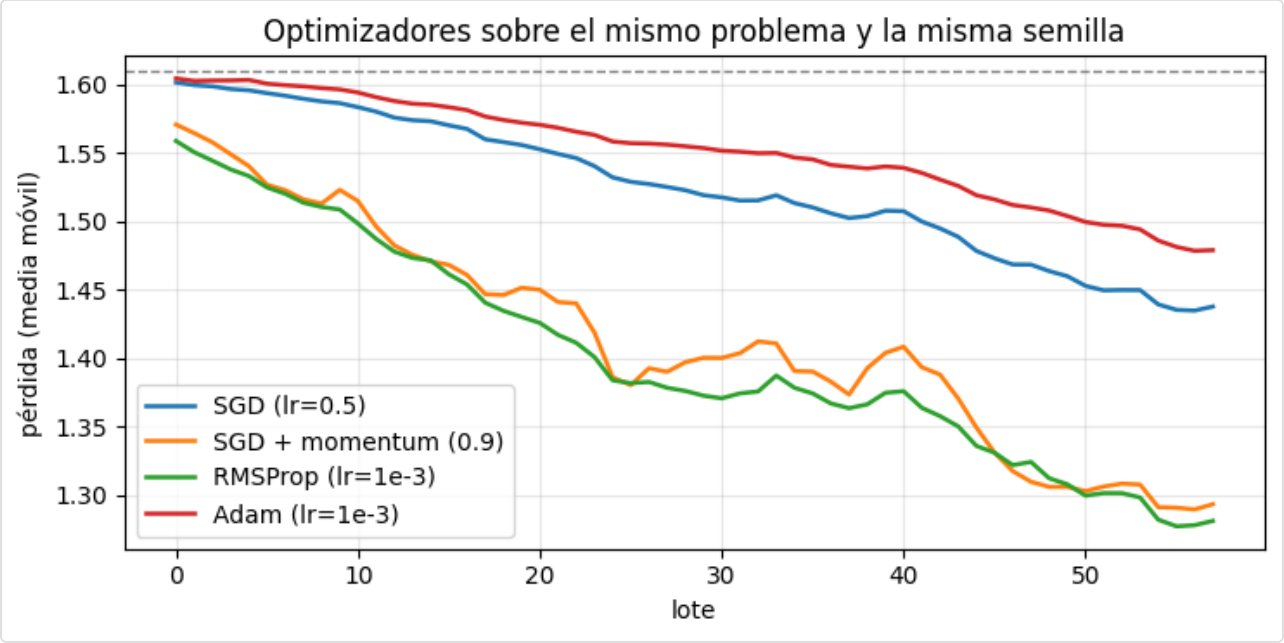
Momentum no es un esquema de actualización (no decide cuántos ejemplos ver por paso); es una variante del optimizador que acumula un promedio móvil de gradientes anteriores — suele aparecer como distractor en este emparejamiento.

> **Cálculo de actualizaciones por época:** 180.000 reseñas / mini-lotes de 64 = 2.812,5 → redondeando hacia arriba, **2.813** actualizaciones. Con full-batch serían 1 por época; con incremental, 180.000.

---

12. Optimizadores

Comparados sobre el mismo problema y la misma semilla: SGD, SGD+momentum(0,9), RMSProp y Adam.



Comparación de optimizadores — en esta corrida particular, momentum y RMSProp bajan más rápido que Adam

> Carácter de cada uno: **SGD** da pasos parejos con la tasa fija. **Momentum** acelera si la dirección del gradiente se sostiene entre pasos (como una bola rodando). Los **adaptativos** (RMSProp, Adam) ajustan el tamaño de paso por parámetro según su historial de gradientes. **Adam** es el más usado por defecto en la práctica. El propio notebook advierte que esta comparación no es "justa": cada optimizador tiene su propia tasa de aprendizaje, así que no se puede concluir en general que uno sea "mejor" que otro solo por esta corrida.

---

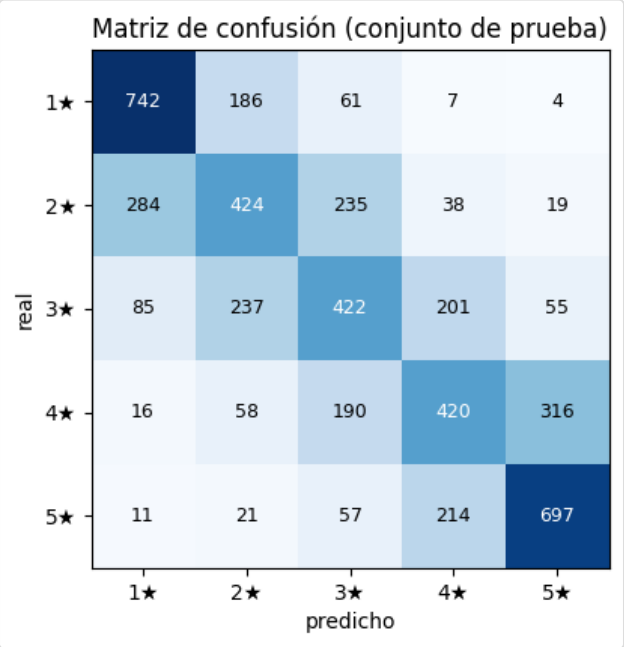
13. Entrenamiento, validación y prueba

Conjunto	Para qué se usa
Entrenamiento (60–80%)	Ajustar los pesos con descenso de gradiente
Validación (10–20%)	Elegir hiperparámetros (épocas, tasa de aprendizaje), decidir cuándo frenar, comparar arquitecturas
Prueba (10–20%)	Reportar la **estimación final** de cómo se comporta el modelo con datos nuevos — se usa una sola vez

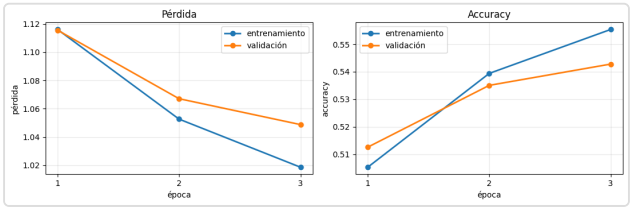
Si las decisiones de diseño se toman mirando el conjunto de prueba, deja de ser una estimación honesta: prueba debe quedar "nunca visto" hasta el final.

---

14. Overfitting, underfitting y regularización



Matriz de confusión del clasificador de estrellas



Curvas de pérdida y accuracy: entrenamiento sigue mejorando, validación se estanca

Cómo se reconoce el sobreajuste (overfitting)

> La pérdida de **entrenamiento** sigue bajando época tras época, mientras la de **validación** se aplana y después empieza a subir. El modelo memoriza los ejemplos vistos y deja de generalizar. En el clasificador de reseñas no es casual: 99,4% de sus parámetros están dedicados a memorizar palabras sueltas (la tabla de embeddings).

Situación	Qué conviene hacer
Error alto incluso en entrenamiento (**subajuste**)	Más datos, o una red más profunda o más ancha (falta capacidad)
Pocos datos etiquetados, no se pueden conseguir más, y el modelo subajusta	*Data augmentation* (aumento de datos)
Error bajo en entrenamiento, alto en datos nuevos (**sobreajuste**)	Regularización: L1/L2 (penalizar pesos grandes) o **dropout**

Con dropout 0,3 + L2: la accuracy de entrenamiento bajó (52,8%), la brecha entre train y val se achicó, pero la accuracy de **validación** también bajó un poco (54,3% → 53,4%). Moraleja del notebook: el objetivo es **maximizar validación**, no minimizar la brecha entre train y val a cualquier costo.

Dropout: qué hace y por qué importa `eval()`

> `nn.Dropout(p)` apaga neuronas al azar **durante el entrenamiento**, para que ninguna se vuelva indispensable y el modelo generalice mejor. En modo `eval()` deja de apagar neuronas y usa la red completa. Olvidarse de llamar `modelo.eval()` antes de medir (y `modelo.train()` antes de seguir entrenando) es un bug difícil de encontrar: da métricas ruidosas sin ningún error visible.

---

15. Métricas: accuracy, precisión, recall, matriz de confusión

$$\text{accuracy} = (\text{aciertos}) / (\text{total}) \quad | \quad \text{precisión} = TP / (TP+FP) \quad | \quad \text{recall} = TP / (TP+FN)$$

Ejemplo de filtro de spam (spam = clase positiva): 13 correos, 8 spam y 5 legítimos. De los 8 spam: 5 detectados (TP), 3 dejados pasar (FN). De los 5 legítimos: 2 marcados como spam (FP), 3 correctos (TN). accuracy = 8/13 = **61,5%** · precisión = 5/(5+2) = **71,4%** · recall = 5/(5+3) = **62,5%**

> **Trampa clásica de accuracy con clases desbalanceadas:** en un conjunto donde el 99% de los correos es legítimo, un filtro que siempre dice "nunca es spam" tiene 99% de accuracy. La métrica que lo delata es el **recall de la clase spam**: de todo el spam real, no atrapó nada, así que vale **0**. La precisión de esa clase directamente no se puede calcular (0/0, nunca marcó nada como spam).

> **Por qué la matriz de confusión dice más que la accuracy sola:** en el clasificador de estrellas, los errores se concentran en celdas **adyacentes** a la diagonal (4 confundido con 5, 2 con 3) y casi nunca en las lejanas (1 con 5). La accuracy "cruda" castiga igual confundir 4 con 5 que 1 con 5; la matriz muestra que el modelo, cuando falla, lo hace entre clases genuinamente parecidas — 91,4% de los errores quedan a lo sumo a una estrella de diferencia.

---

16. Guardar y cargar modelos

> `torch.save(modelo.state_dict(), "clasificador.pt")` guarda solo un **diccionario con los tensores de los parámetros**, no la arquitectura ni la clase. Para volver a usarlo en otra sesión: (1) construir el modelo con la **misma arquitectura**, (2) cargarle los pesos con `modelo.load_state_dict(torch.load(...))`, y (3) pasarlo a `modelo.eval()` antes de predecir (relevante en cuanto haya dropout o batch norm). `torch.load` solo no alcanza: no devuelve un modelo listo, devuelve el diccionario de tensores.

---

17. Embeddings aprendidos: qué queda al final

Terminado el entrenamiento del clasificador de reseñas, los vectores de la tabla de embeddings funcionan como un **léxico de sentimiento**: si se los interpreta como "estrella promedio", `excelente` da 5,00★, `pésimo` 1,00★, `recomiendo` 4,97★, `devolver` 1,01★. Pero los vecinos por similitud de coseno

son ruidosos, porque la tarea solo entrena un único eje malo↔bueno, no un mapa semántico general (eso es lo que la Unidad 2, con la hipótesis distribucional, resuelve mejor).

UNIDAD 2 — Secuencias, memoria y atención

Recorrido de la unidad: cómo obtener vectores de palabras sin anotar nada (embeddings distribucionales) → redes recurrentes para procesar secuencias → modelado de lenguaje y generación de texto → traducción con encoder-decoder → el cuello de botella del vector único → atención, que lo resuelve.

Embeddings distribucionales

De la tabla de rasgos a la hipótesis distribucional

Antes de los embeddings aprendidos automáticamente, una alternativa era anotar a mano una tabla de rasgos (¿se bebe? ¿es un aparato? ¿polaridad? ¿habla de dinero?) para cada palabra.

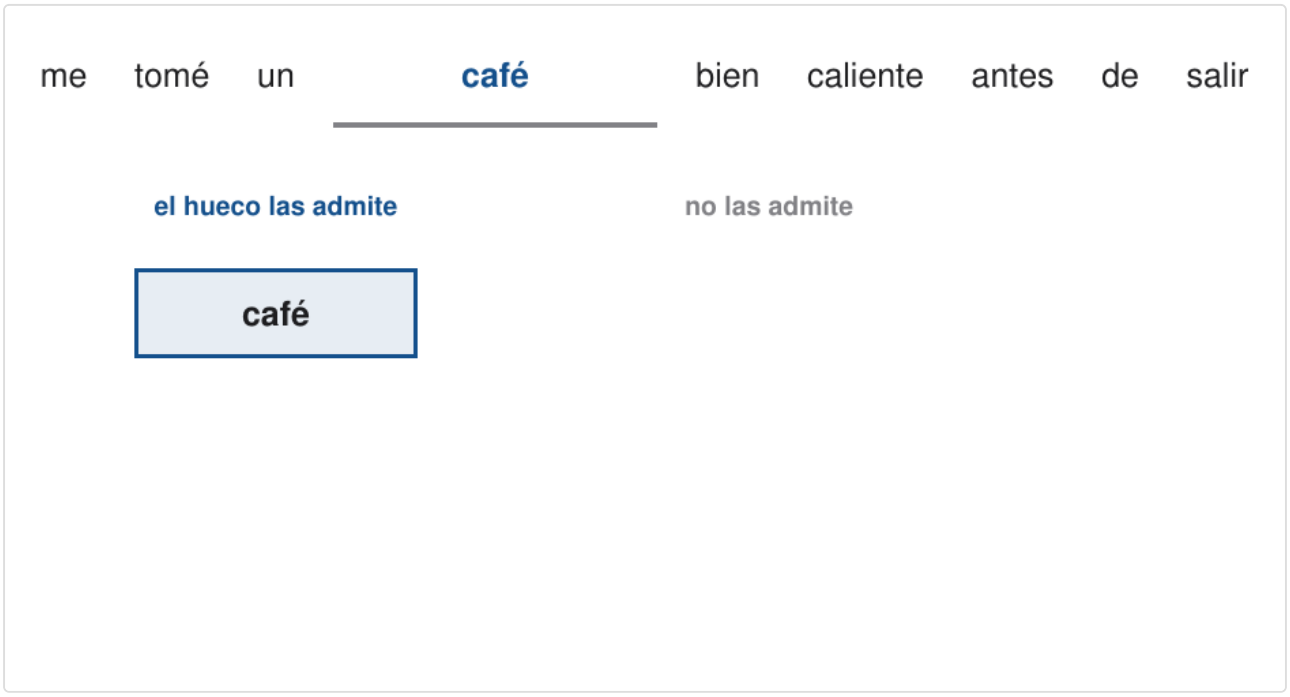
	se bebe	es un aparato	polaridad	habla de dinero	
café	1,0	0,0	0,3	0,0	dos filas casi iguales cos = +0,996
té	1,0	0,0	0,2	0,0	
teléfono	0,0	1,0	0,0	0,2	
batería	0,0	1,0	0,0	0,0	
excelente	0,0	0,0	1,0	0,0	dos filas opuestas cos = −1,000
pésimo	0,0	0,0	−1,0	0,0	
caro	0,0	0,0	−0,3	1,0	

Tabla de rasgos anotados a mano: cos(café, té) = 0,996; cos(excelente, pésimo) = −1,000; cos(café, teléfono) = 0

> No se hace así en la práctica por tres razones: el **costo** de anotar a mano, la falta de un **criterio** objetivo para elegir qué columnas incluir, y que los rasgos relevantes pueden no ser **nombrables** con palabras.

La hipótesis distribucional

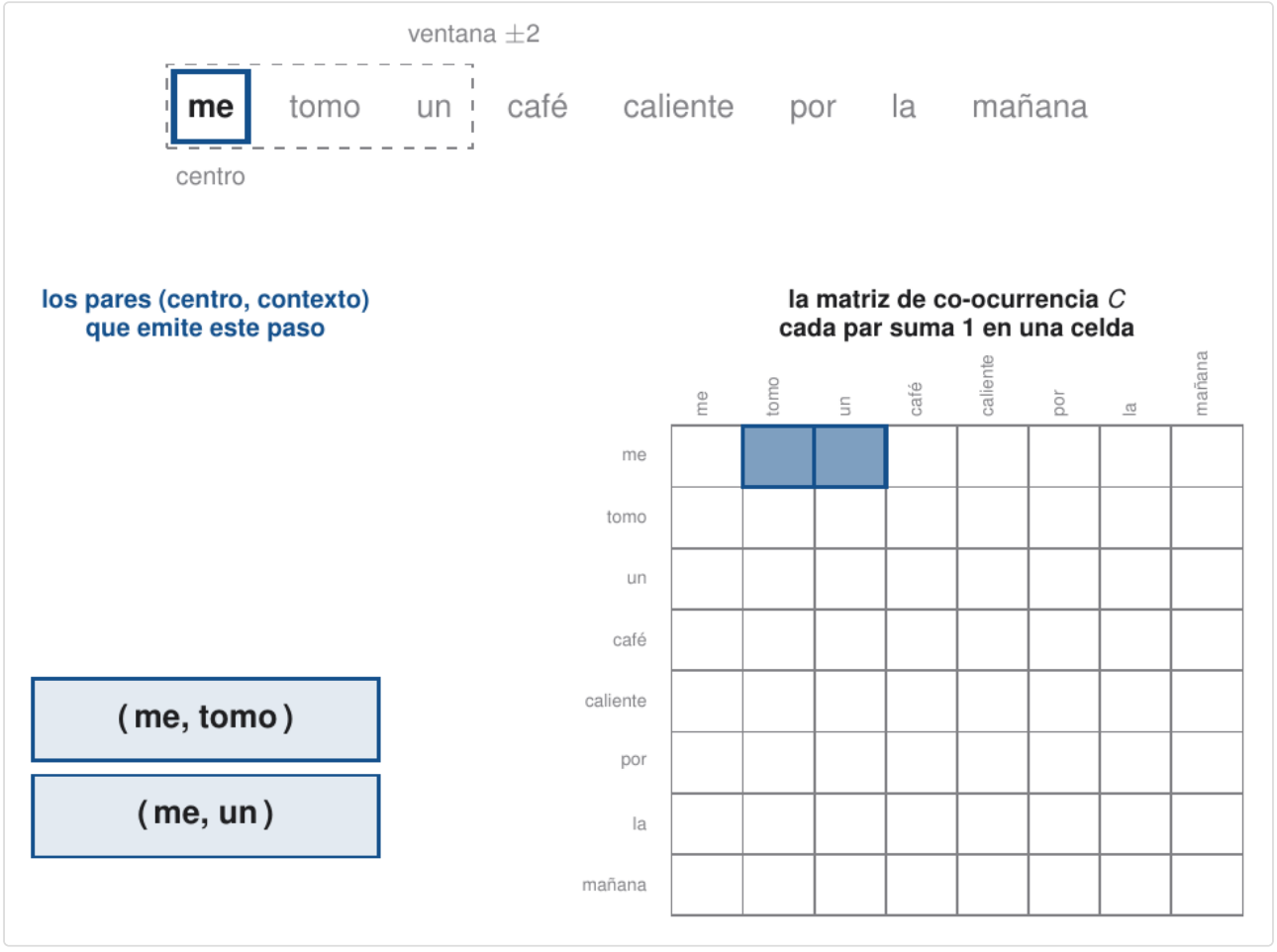
> "Palabras que aparecen en contextos parecidos tienen significados parecidos" (Harris, Firth)



"me tomé un ___ bien caliente" admite café, té, mate — no admite teléfono

Es **operativa**: a diferencia de la tabla de rasgos, el contexto se puede contar automáticamente, sin anotar nada a mano.

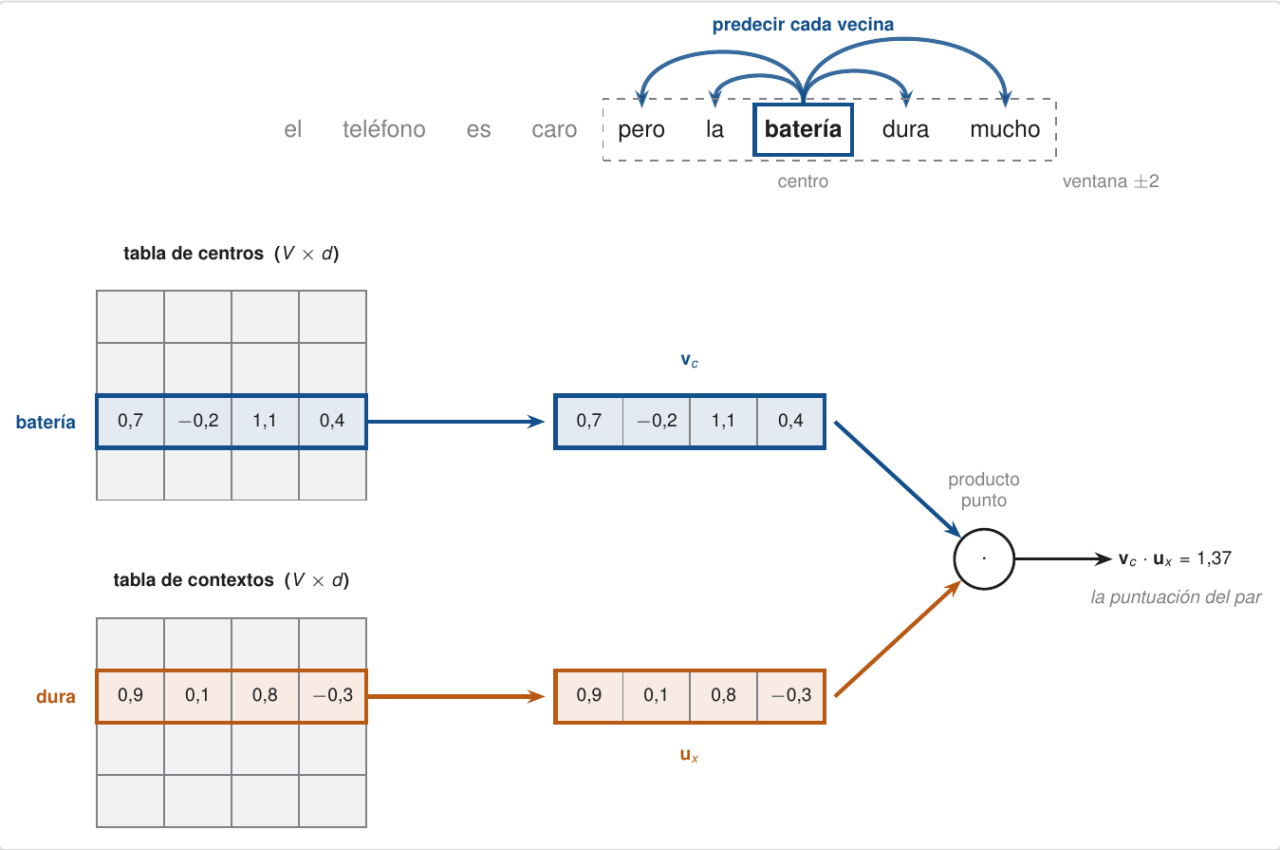
Ventana, co-ocurrencia y skip-gram



Ventana ±2, pares (centro, contexto) y la matriz de co-ocurrencia C

> Se define una **ventana** de tamaño ±m alrededor de cada palabra central. La matriz de co-ocurrencia C (V×V) cuenta, para cada par (i, j), cuántas veces j aparece dentro de la ventana de i. **La fila i de C es el vector de la palabra i.**

Skip-gram



Tarea skip-gram: predecir cada palabra de la ventana a partir de la central; dos tablas (centros y contextos)

> **Tarea:** dada la palabra central, predecir las palabras de su ventana. Es **autosupervisada** (la etiqueta sale del texto mismo, nadie anota nada). Tiene **dos tablas** V×d: una de centros (v) y una de contextos (u). La puntuación de un par es **s(c, x) = v_c · u_x** (producto punto). No hay capas ocultas ni no linealidades: es el modelo más simple posible.

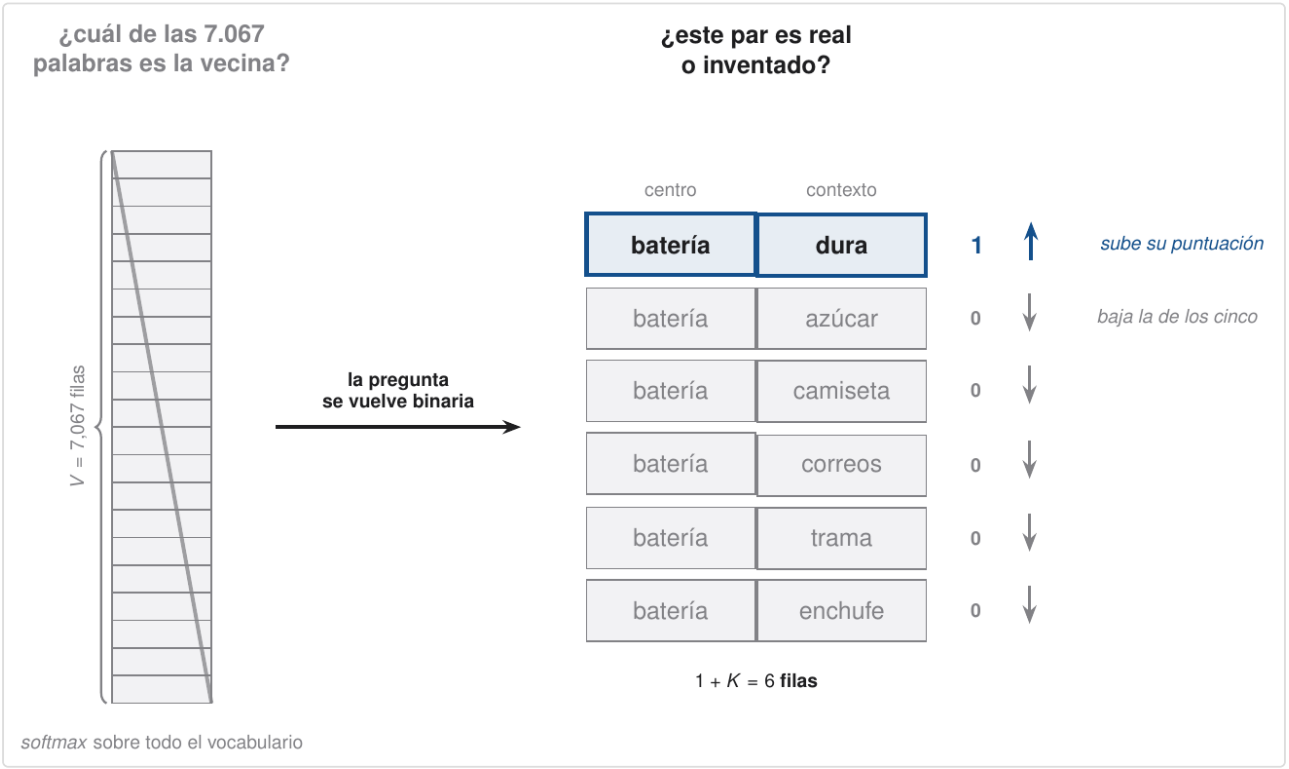
Por qué dos tablas y no una: con una sola tabla, una palabra tendría incentivo permanente de inflar su propia norma (v_w · v_w = ‖v_w‖²), lo que rompe el entrenamiento. Con dos tablas separadas eso no ocurre.

Pérdida: softmax sobre todo el vocabulario V + entropía cruzada.

> **El vector final es un subproducto:** al terminar el entrenamiento se descarta el "clasificador" (la tarea de predecir el contexto) y la tabla de contextos; se conserva solo la tabla de **centros**, que es el embedding final de cada palabra.

CBOW es la tarea inversa: predice la palabra central a partir de su ventana. Entrena más rápido y funciona mejor con palabras frecuentes. **Skip-gram** funciona mejor con palabras raras y corpus chicos.

Muestreo negativo

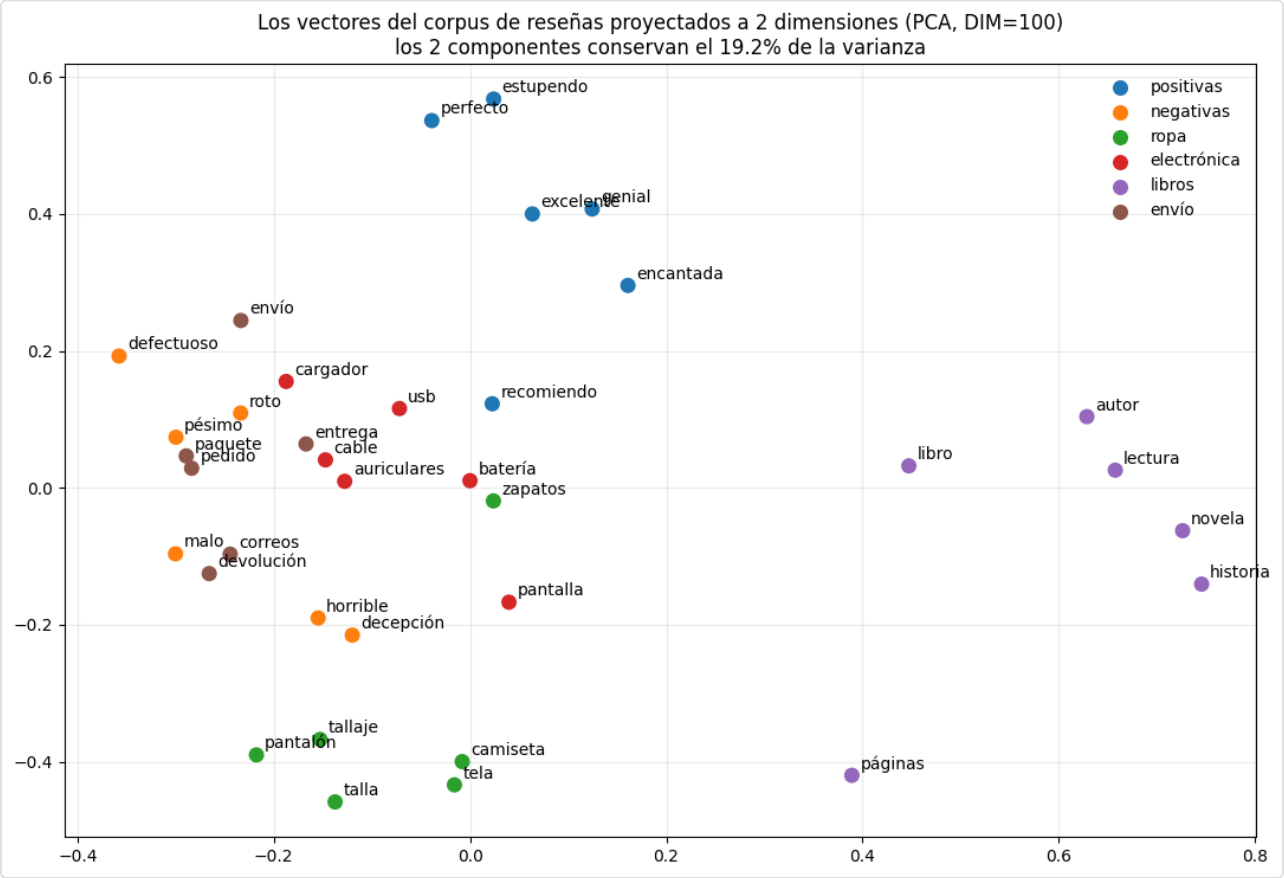


De softmax sobre 7.067 palabras a una pregunta binaria con 1+K=6 filas

> **Problema:** el softmax sobre todo el vocabulario es carísimo (del orden de 10^{10} productos punto por época en corpus reales). **Solución:** convertir la tarea en una pregunta **binaria** ("¿este par centro-contexto es real o inventado?"), resuelta con **sigmoide**. Por cada par real se sortean K = 5 pares falsos (negativos), o sea 1+K = 6 filas en vez de V filas.

Los negativos se sortean con probabilidad P_neg ∝ f(w)^(3/4), donde f(w) es la frecuencia de la palabra. Este exponente **aplana** la distribución de frecuencias: en el ejemplo del notebook, una palabra muy común como "de" sale sorteada solo 0,41 veces lo que le tocaría por frecuencia pura, y una palabra rara como "tallaje" sale 2,07 veces más. Con frecuencia real pura, casi todos los negativos serían palabras funcionales triviales ("de", "que", "la"); con distribución uniforme, casi todos serían rarísimas — ambos extremos hacen la tarea trivial y no entrenan nada útil.

Gráfico PCA de los vectores entrenados



Proyección a 2D (PCA) de vectores de 100 dimensiones — solo conserva 19,2% de la varianza

> Los grupos temáticos (libros, ropa, electrónica) se distinguen razonablemente bien; las palabras de envío quedan mezcladas entre las reseñas negativas (nadie habla del envío cuando todo salió bien).
> **Advertencia clave:** la proyección a 2D descarta información — acá conserva solo 19,2% de la varianza total —, así que lo que se ve es un **indicio**, no una prueba concluyente de la calidad de los embeddings.

Límites de la hipótesis distribucional

> **Similitud distribucional ≠ mismo tema:** "teléfono" y "batería" pueden dar coseno ≈ 0 en un corpus chico simplemente porque nunca coincidieron en la misma ventana, aunque estén relacionados semánticamente.
> **Antónimos quedan cerca:** "mucho"/"poco" pueden dar coseno cercano a 1, y "caro"/"barato" también (0,919 en corpus de juguete, 0,592 en reseñas reales) — a veces incluso más cerca que pares sinónimos como batería/cargador (0,477). Es un límite estructural de la hipótesis (aparecen en los mismos contextos, aunque signifiquen lo opuesto), no se arregla juntando más datos.
> **Una palabra, un solo vector:** "banco" (asiento / institución financiera) tiene un único vector que promedia todos sus sentidos.
> Los embeddings también **heredan los sesgos** presentes en el corpus de entrenamiento.

word2vec, GloVe y fastText

Modelo	Año	Idea

word2vec	2013	**Predictivo**: skip-gram / CBOW, entrena prediciendo el contexto
GloVe	2014	Usa la matriz de co-ocurrencia **global** del corpus; ajusta v·u ≈ log(co-ocurrencia)
fastText	2016	Usa **n-gramas de caracteres** (3 a 6); puede dar vector a palabras nunca vistas

---

Clase 1 — Redes recurrentes (RNN)

Por qué el promedio de embeddings no alcanza

> El clasificador de la Unidad 1 promedia los embeddings de todas las palabras: **tira el orden**. Al permutar las palabras de una reseña, el modelo de bolsa (promedio) da la **misma** predicción en 200 de 200 casos probados (diferencia ~10^{-8}): es una propiedad matemática del promedio, no un defecto de entrenamiento. Una red recurrente, en cambio, cambia su predicción en 49 de 200 casos ante la misma permutación: sí es sensible al orden.

RNN a mano: fórmula y costo

$$h_t = \tanh(x_t \cdot W_{ih}^T + b_{ih} + h_{t-1} \cdot W_{hh}^T + b_{hh})$$

```
h = torch.zeros(B, oculta)
for t in range(T):
    x_t = x[:, t]                       # (B, entrada)
    h = torch.tanh(x_t @ W_ih.T + h @ W_hh.T + b)
estado_final = h                        # (B, oculta)
```

> El `for` recorre las **posiciones de la secuencia** (el tiempo), procesando las B secuencias del lote juntas en cada paso. El paso t **necesita** el resultado del paso t−1, así que este bucle **no se puede paralelizar en el tiempo** (sí se paralelizan las B secuencias del lote entre sí). Esta limitación es justamente lo que después motiva a los transformers.

Parámetros de una RNN con 2 entradas y 3 unidades: W_ih (3×2=6) + b_ih (3) + W_hh (3×3=9) + b_hh (3) = **21** parámetros, contra 9 de una capa lineal simple de igual tamaño (2→3): los 12 extra son toda la memoria del modelo.

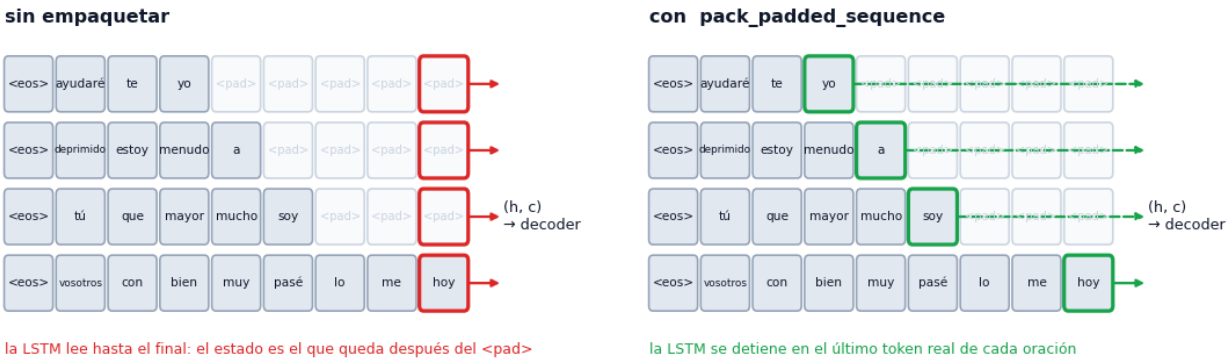
Arquitecturas según la tarea

Tarea	Esquema
Clasificar una reseña con una cantidad de estrellas	**Muchos a uno** — secuencia de entrada, una sola salida
Describir una imagen con una oración	**Uno a muchos** — una entrada, secuencia de salida
Etiquetar cada palabra con su categoría gramatical	**Muchos a muchos, alineado** — una salida por cada entrada, en el mismo paso

Traducir una oración entera a otro idioma	**Muchos a muchos, diferido (encoder-decoder)** — hay que leer toda la entrada antes de producir nada

> **"Diferido" significa** que el modelo no empieza a producir salida hasta haber **leído la entrada entera**. Eso le permite reordenar, condensar o expandir según lo pida el idioma destino — algo que un esquema alineado (uno a uno, en el mismo orden) no puede hacer, porque no es lo mismo traducir español→inglés que inglés→español: la cantidad y el orden de elementos cambian.

Relleno en secuencias: el problema de no avisar dónde termina cada una



Sin empaquetar, el estado final incluye el relleno; con pack_padded_sequence, la LSTM se detiene en el último token real

> Un clasificador recurrente que decide la clase a partir del **estado final** de la LSTM tiene un problema serio si el tensor rellenado entra "tal cual", sin avisarle a la capa dónde termina cada reseña: la **misma reseña** da un estado final **distinto** según cuánto relleno le tocó en su lote — su predicción depende de con quién viajó. La solución es `pack_padded_sequence`, que le pasa a la capa los largos reales y hace que se detenga exactamente en el último token real de cada secuencia (diferencia medida: 0,317 en los logits sin empaquetar, 0 con `pack_padded_sequence`).

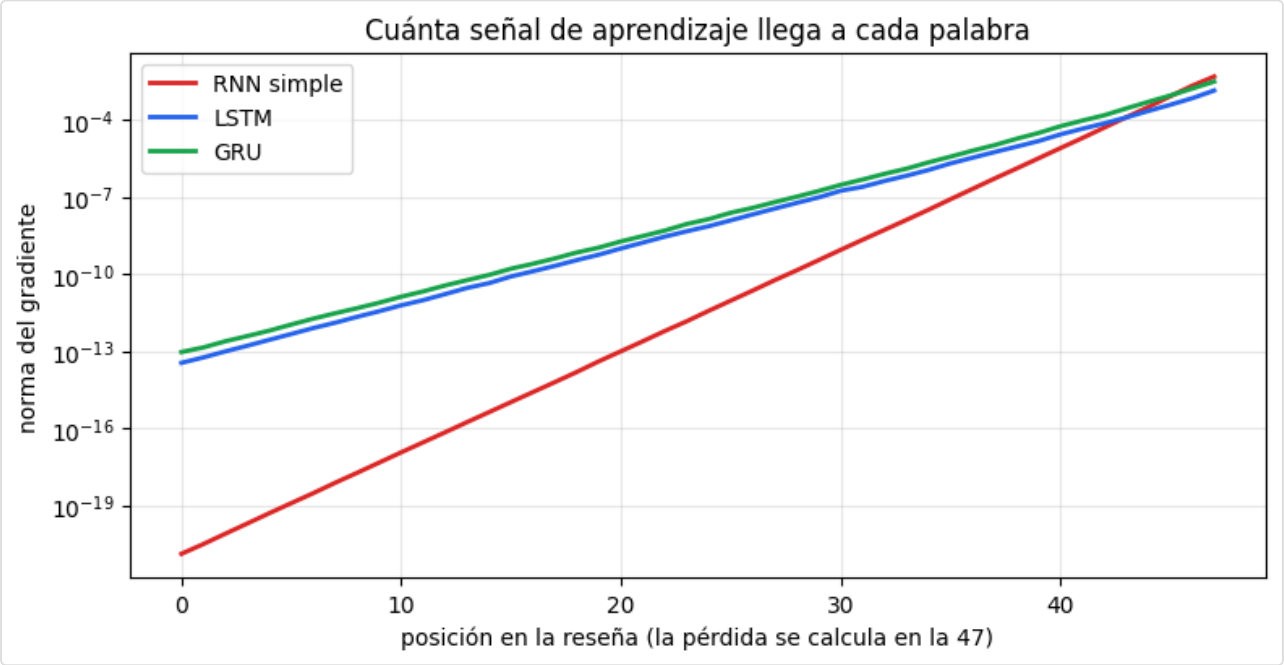
LSTM empaquetada: formas de salida

```
salidas, (h, c) = lstm(entrada_empaquetada)   # LSTM de 1 capa, 128 unidades
```

Tensor	Forma	Qué es
`salidas`	(B, L, 128)	Un vector de estado **por posición** de la secuencia (el h de cada paso)
`h`	(1, B, 128)	Una fila **por capa**; es el estado final
`c`	(1, B, 128)	**Memoria interna** de la celda: viaja de paso en paso y no aparece en `salidas`

> Para clasificar la reseña se usa `h[-1]`: el estado después de la **última palabra real** (con `pack_padded_sequence`, ya está bien calculado). Usar `salidas[:, -1]` (la última posición del tensor) es la trampa típica: esa posición puede ser puro relleno si la reseña es más corta que el máximo del lote. RNN y GRU devuelven solo `h` (no tienen memoria de celda separada).

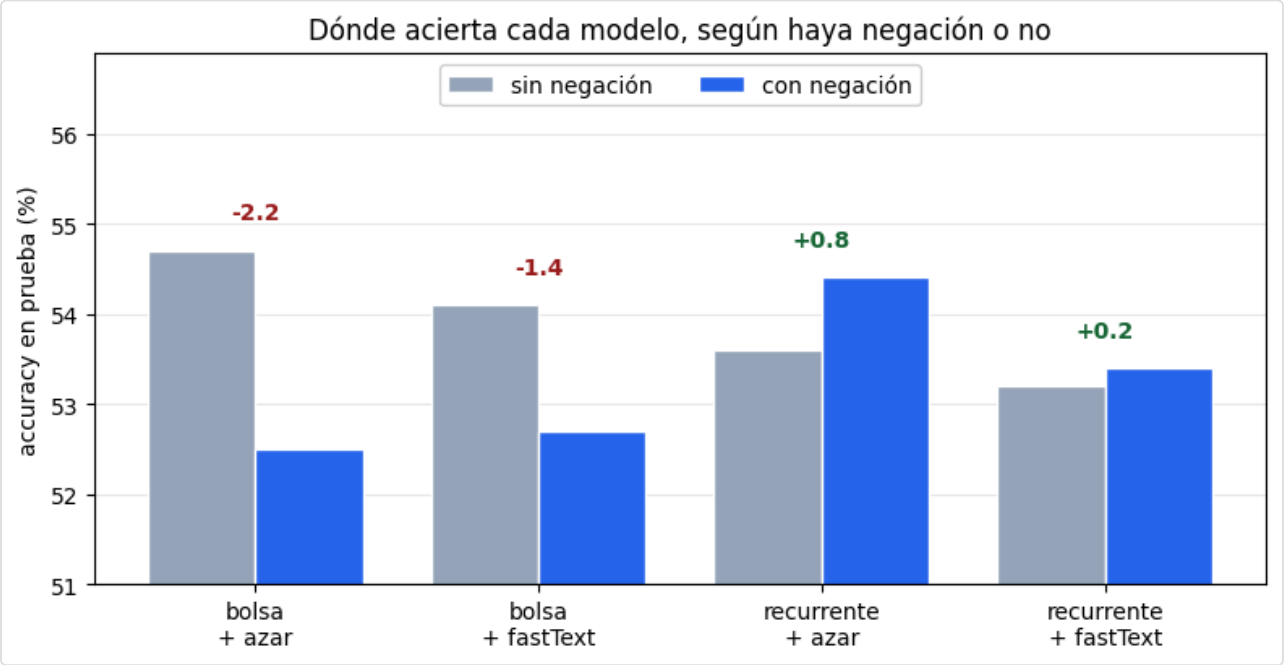
Desvanecimiento del gradiente: RNN vs. LSTM vs. GRU



Norma del gradiente que llega a cada posición (escala log): RNN simple vs. LSTM vs. GRU

> Un clasificador que decide la clase mirando solo el estado de la **última** posición (donde se calcula la pérdida) necesita que el gradiente viaje hacia atrás por todos los pasos de la recurrencia para que las primeras palabras influyan en algo. Medido en el notebook, en la posición 0 la RNN simple recibe un gradiente **ocho órdenes de magnitud menor** que la LSTM (un 1 seguido de ocho ceros de diferencia). **Implicancia:** en la RNN simple, la primera palabra de una reseña larga prácticamente no participa del aprendizaje. La LSTM **mitiga** el desvanecimiento gracias a sus compuertas (el estado de celda se multiplica por la compuerta de olvido y se le suma lo nuevo, así que si la compuerta está cerca de 1 la información y el gradiente cruzan el paso casi intactos) — pero no lo **elimina**, ni permite recordar secuencias de cualquier largo. Es mitigación, no cura.

Negación: dónde el orden sí importa



Accuracy con y sin negación por modelo: en bolsa el signo es negativo, en recurrente es positivo

> Comparando accuracy con y sin negación en la oración, en los modelos de **bolsa** (promedio) el efecto es **negativo** (−2,2 y −1,4 puntos), y en los **recurrentes** es **positivo** (+0,8 y +0,2): el signo se invierte. La accuracy global es casi idéntica entre los cuatro modelos — la diferencia real no está en el promedio, sino en **dónde** cada arquitectura gana o pierde: la recurrente gana específicamente donde el orden de las palabras cambia el significado.

Costo comparado de las celdas recurrentes

Cada celda tiene distinta cantidad de "juegos" de pesos: la RNN simple uno, la GRU tres, la LSTM cuatro. En el notebook (dim entrada=300, oculta=128): RNN simple 55.040 parámetros, GRU 165.120, LSTM 220.160.

Clase 2 — Modelado de lenguaje

Autosupervisión: de dónde salen las etiquetas

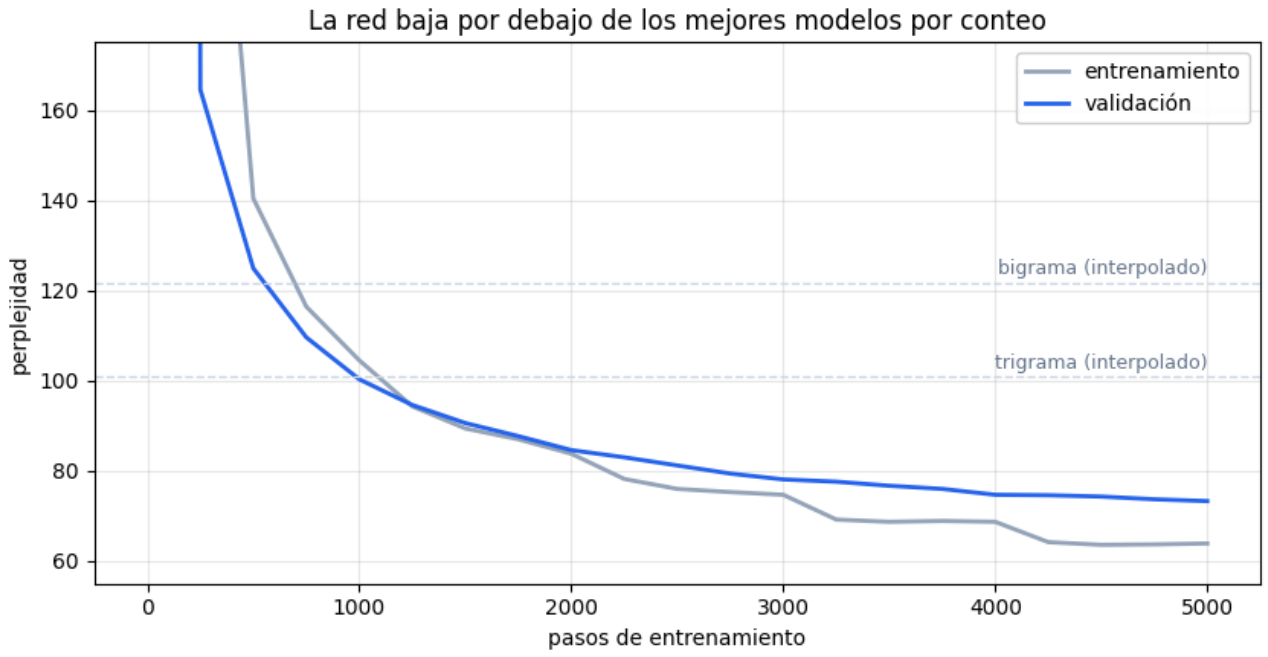
> Para entrenar el clasificador de reseñas hizo falta que alguien anotara la cantidad de estrellas. Para entrenar un modelo de lenguaje alcanza con **texto suelto** (cuentos, noticias, páginas web) sin que nadie lo anote. Las etiquetas salen del **texto mismo, corrido un lugar**: la entrada es la secuencia y el objetivo es esa misma secuencia desplazada una posición. Nadie etiqueta nada, y por eso escala a cantidades enormes de texto (cada oración de 20 tokens produce 19 preguntas con respuesta). La capa de salida tiene una unidad por palabra del vocabulario, y sigue habiendo función de pérdida (entropía cruzada) como en cualquier clasificador — no es "sin pérdida". El split se hace por **oración**, para evitar fugas entre ventanas de entrenamiento y validación.

Perplejidad

$$\text{perplejidad} = \exp(\text{entropía cruzada media}) \quad = \quad \exp(-(1/N)\ \Sigma\ \ln P(w_i \mid \text{contexto}))$$

Se lee como **"entre cuántas palabras duda el modelo, en promedio"**.

Modelo uniforme (reparte la probabilidad por igual entre V palabras): perplejidad = **V exacto**. Con vocabulario de 5.003 tokens → perplejidad = **5.003**. Un modelo **sin entrenar** da un valor muy cercano a V (pérdida 8,522 contra ln(5.003) = 8,518).
De entropía cruzada a perplejidad: con entropía cruzada media 4,29 → perplejidad = exp(4,29) ≈ **73**.



La LSTM cruza por debajo del unigrama, el bigrama y el mejor trigrama

Modelo	Perplejidad (validación)
Uniforme	5.003
Unigrama	432,6
Bigrama (Laplace)	270,8
Trigrama (Laplace)	1.240,8 (¡peor que el unigrama!)
Bigrama interpolado	121,5
Trigrama interpolado	100,8
LSTM	**73,1**

Por qué el trigrama con Laplace es peor que el unigrama: el 39% de los trigramas de validación nunca se vieron en entrenamiento — el modelo de n-gramas es demasiado disperso (sparse). La LSTM gana porque **generaliza** a combinaciones nuevas, no porque "mire más contexto": entre 16 y 128 tokens de contexto la perplejidad baja apenas 4 puntos (de hecho, usa poco más de una oración de contexto real).

Arquitectura del modelo de lenguaje

Embedding → LSTM → Linear(V) → salida (B, L, V), un logit por palabra en cada posición

Reparto de parámetros: tabla de **salida** 55%, LSTM 17%, tabla de **entrada** 28%. Se usa `clip_grad_norm_` porque, a diferencia del desvanecimiento, el gradiente también puede **explotar** (crecer sin control) al entrenar secuencias largas.

Por qué la tabla de salida "aprende español" y la de entrada no: la fila de **salida** de cada palabra recibe gradiente en **todas sus filas, en todos los pasos** (la softmax sobre 5.003 palabras las compara y empuja a todas — 5.003 de 5.003 filas actualizadas en un lote). La tabla de **entrada** solo actualiza la fila de la palabra que efectivamente aparece en ese lote (en el notebook, 718 de 5.003). Por eso, buscando por coseno las filas más parecidas a "rojo": en la tabla de **salida** aparecen negro, verde, blanco, rubio (relación semántica real); en la de **entrada**, palabras sin relación entre sí. Esto motiva el *weight tying* (compartir la misma tabla para entrada y salida) en modelos modernos.

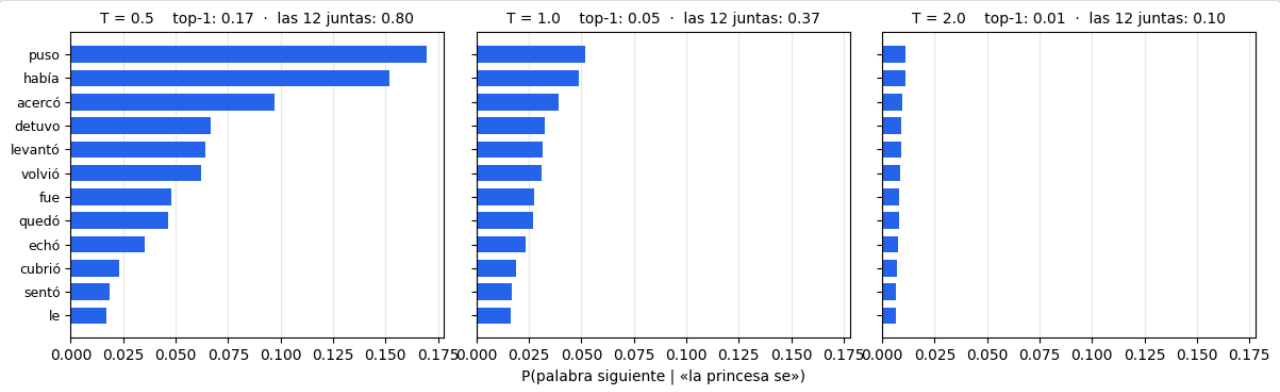
Generación de texto

Paso 1: alimentar la semilla y obtener los logits de la palabra siguiente
Paso 2: aplicar las "perillas" (temperatura, top-k o top-p) y sortear una palabra de la distribución
Paso 3: agregar la palabra elegida a la salida
Paso 4: realimentar esa palabra como próxima entrada, con el estado de la LSTM a cuestas
Paso 5: repetir hasta el largo pedido o hasta el token <eos>

Estrategia voraz: por qué repite en bucle

Elegir siempre la palabra más probable (argmax) es **determinista**: si el proceso vuelve a un estado ya visitado, repite exactamente el mismo camino **para siempre**. Esto pasa tanto con un bigrama por conteo como con una LSTM — la LSTM tarda más en caer en el bucle (tiene más memoria), pero cae igual. La solución es **sortear** de la distribución en vez de tomar el máximo.

Temperatura

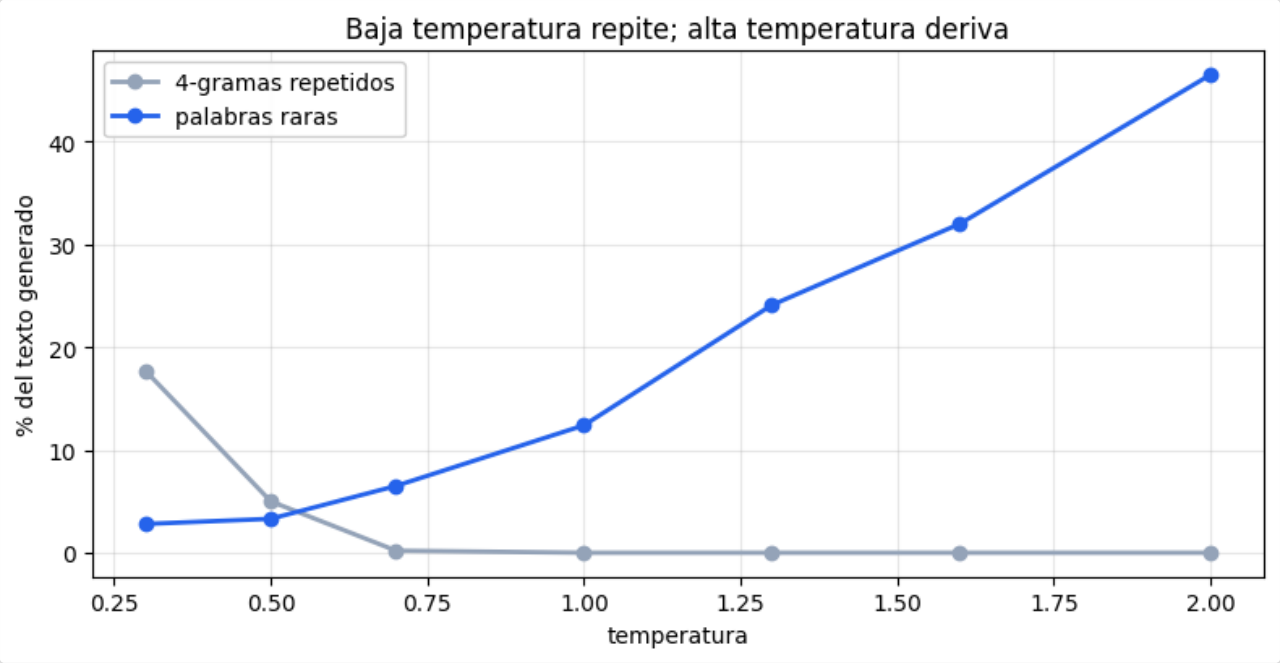


Tres temperaturas sobre "la princesa se": T=0,5 afila, T=1 normal, T=2 aplana

T < 1 afila la distribución (más determinista); T > 1 la aplana (más aleatoria). En el ejemplo, T=0,5 hace que las 12 palabras top sumen 0,80 de probabilidad; T=1, 0,37; T=2, apenas 0,10.

Top-k y top-p

Top-k conserva siempre las *k* palabras más probables, sea el modelo esté seguro o indeciso. **Top-p** (nucleus sampling) conserva las palabras más probables hasta acumular una probabilidad p; se **adapta** al contexto: con distribución repartida (el modelo duda entre cientos de continuaciones razonables) conserva muchas palabras (654 en el ejemplo); con distribución concentrada (pocas continuaciones razonables) conserva pocas (188). Top-k=10 conserva siempre 10, sea cual sea el contexto — donde el modelo está seguro, top-k puede dejar entrar candidatas malas; donde duda genuinamente entre cientos, top-k excluye opciones válidas. Por eso las APIs actuales exponen top-p combinado con temperatura.

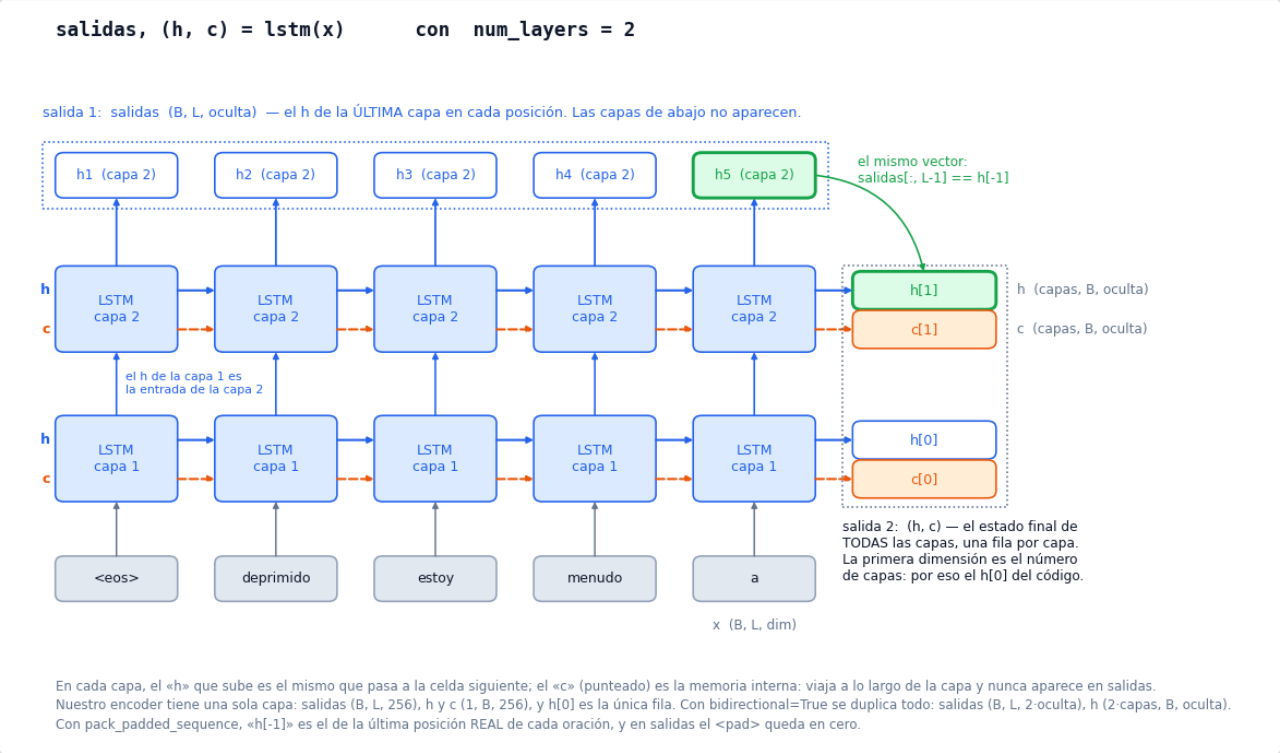


Compromiso: baja temperatura repite 4-gramas, alta temperatura genera palabras raras

T=0,3: 17,7% de 4-gramas repetidos, 2,8% de palabras raras. T=2: 0% repetidos, 46,5% raras. No hay una T "correcta" universal; la zona útil en este corpus está entre 0,7 y 1.

Clase 3 — Traducción: encoder-decoder

Encoder: qué le pasa al decoder

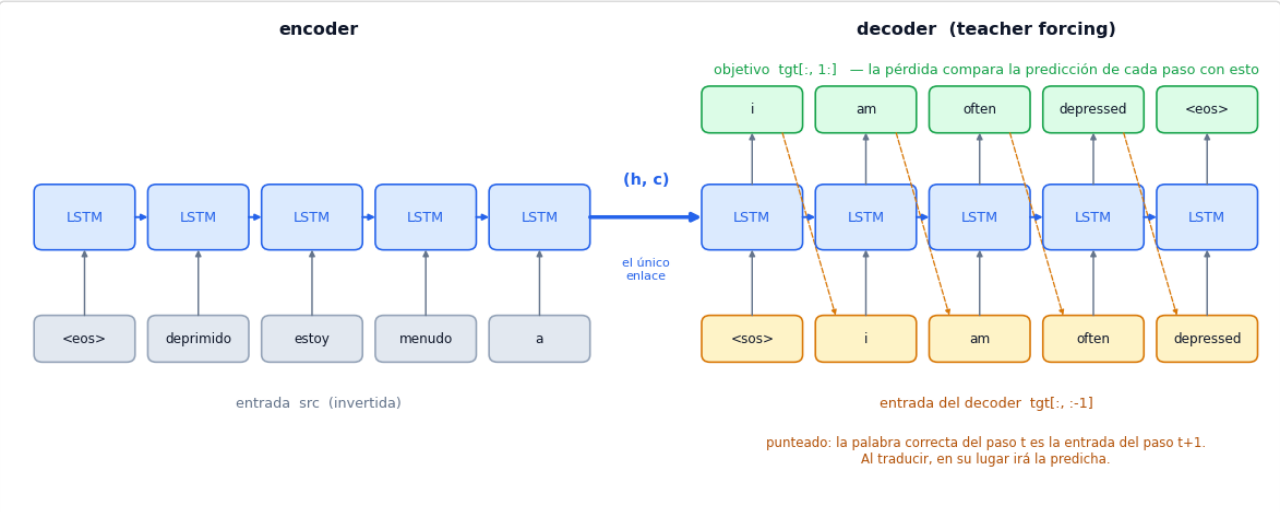


salidas (B,L,oculta): el h de la última capa en cada posición; h y c: el estado final de cada capa

> Encoder recibe (B, L_es) → devuelve al decoder solo el estado final (h, c), dos tensores de forma (1, B, oculta) — con `num_layers` >1, la primera dimensión es la cantidad de capas

La entrada del encoder se alimenta **invertida** (idea del paper original de Sutskever et al.), para que las primeras palabras del idioma fuente queden más cerca temporalmente de las primeras del idioma destino.

Teacher forcing



El único enlace entre encoder y decoder es (h, c). Entrada del decoder: tgt[:,:-1]; objetivo: tgt[:,1:]

`tgt` es la traducción de referencia, forma (B, L_en), empieza con `<sos>` y termina con `<eos>`. Con teacher forcing, el decoder recibe como entrada `tgt[:, :-1]` (la palabra **correcta** de la referencia, corrida un lugar) y sus logits tienen forma **(B, L_en−1, V_en)**. La pérdida se calcula contra `tgt[:, 1:]` con `ignore_index=0` en la entropía cruzada, para que las posiciones de `<pad>` no cuenten. El tamaño del vocabulario inglés determina el **ancho de la Linear final** del decoder, que es la capa que más pesa (en el notebook, V_en=11.286 filas — más que el vocabulario de entrada, que solo agranda la tabla de embeddings del encoder).

El precio del teacher forcing (sesgo de exposición): en **inferencia**, el decoder recibe su propia predicción del paso anterior, no la palabra correcta. Como durante el entrenamiento siempre se lo guio con la referencia, nunca aprendió a **recuperarse** de un error propio. Si la primera palabra generada en inferencia sale mal, la pérdida efectiva del resto de la oración sube ~20% (1,969 → 2,370 en el notebook).

BLEU



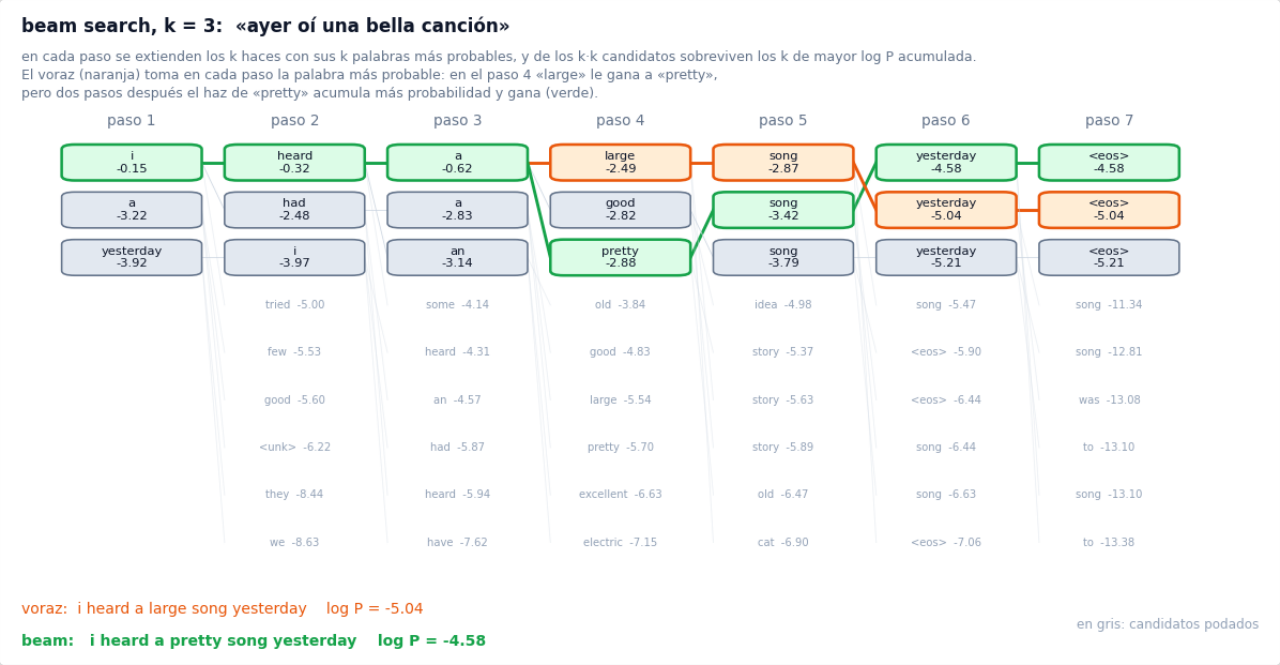
BLEU = brevedad × media geométrica de p1..p4 (precisión de 1- a 4-gramas, con recorte)

$$\text{BLEU} = \text{penalización_de_brevedad} \times \text{media_geométrica}(p_1, p_2, p_3, p_4)$$

Cuenta n-gramas de 1 a 4 con **recorte** (cada n-grama cuenta como máximo las veces que aparece en alguna referencia), y aplica una **penalización por brevedad**: si la hipótesis es más corta que la referencia más cercana, se castiga aunque las precisiones sean perfectas — "lo que falta también castiga". Se calcula a nivel de **corpus** y admite **varias referencias** por oración. Va de 0 a 100.

Ejemplo del notebook: "i am going to bed" da precisiones 4/5, 3/4, 2/3, 1/2 → media geométrica 66,9, brevedad 1,00 → BLEU = **66,9**. "i am going to" (sin errores) tiene precisiones perfectas pero brevedad 0,78 (más corta que la referencia) → BLEU = **77,9** en ese ejemplo puntual, pero con otras referencias más específicas la brevedad puede hundir mucho más el número — el punto es que **una hipótesis "perfecta pero corta" no gana BLEU alto automáticamente.** Resultado real del traductor: BLEU = 26,2 (precisiones 56,0/33,1/20,3/13,0).

Beam search

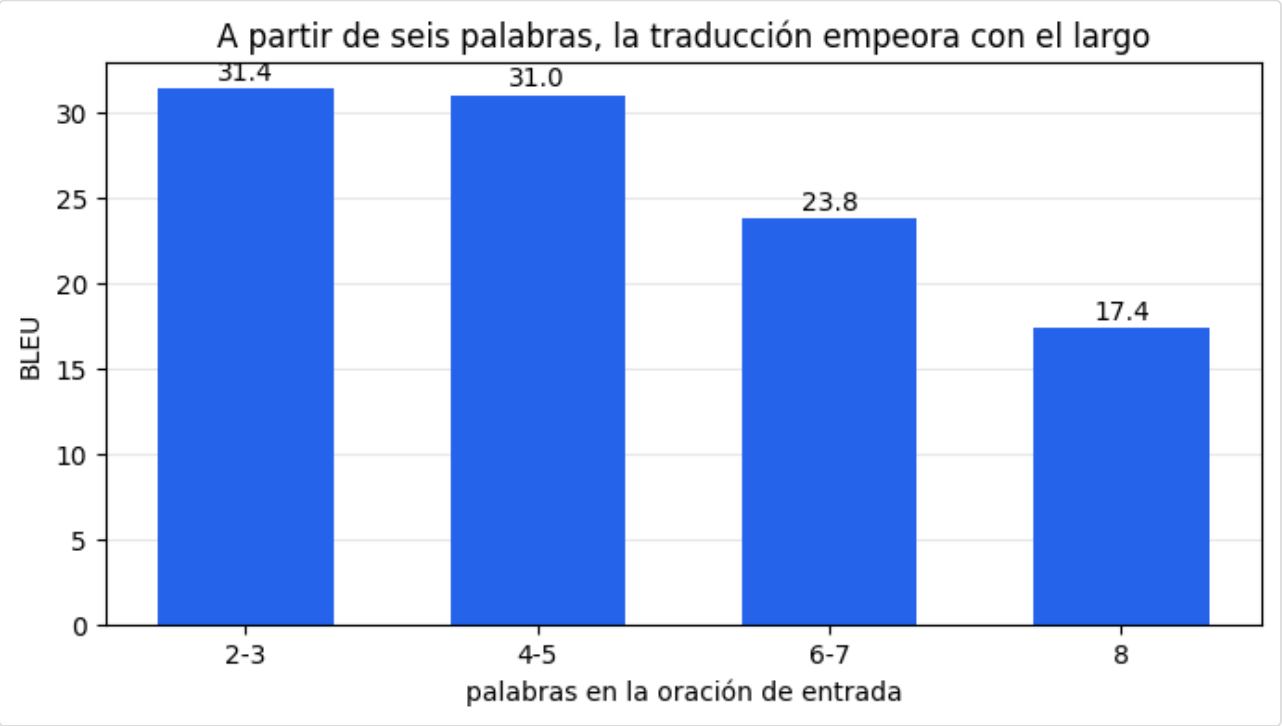


Beam search k=3: el voraz elige "large" en el paso 4, pero "pretty" gana con el haz completo

Por qué elegir la palabra más probable en cada paso no garantiza la traducción más probable: un token muy probable ahora puede llevar a un camino **peor** después. El voraz solo mira un token a la vez; beam search conserva **k secuencias parciales** (el "haz") y las va extendiendo todas, quedándose en cada paso con las k de mayor probabilidad acumulada (log P), en vez de comprometerse con una sola elección local.

En el ejemplo, el voraz elige "large" (log P total −5,04); el haz completo con "pretty" da −4,58 (mejor). Beam cambia la mitad de las traducciones del corpus y sube BLEU de ~27,1 a ~29,6. Como beam favorece secuencias más cortas (menos términos que multiplicar en la probabilidad), se corrige dividiendo por el largo^α (con α≈0,7).

El cuello de botella del vector único



BLEU cae fuerte a partir de seis palabras de entrada

En un encoder-decoder sin atención, **todo** lo que el decoder sabe de la entrada tiene que caber en un **único vector**, el estado final (h, c) del encoder — con entradas largas se pierde detalle. Con oraciones de 2-3 palabras BLEU es 31,4; con 8 palabras cae a 17,4. No es que el decoder "no pueda generar oraciones más largas que la entrada" ni que la LSTM "olvide el principio" (eso se arreglaría con GRU, pero no es la causa real): el problema es de **capacidad del vector**, y por eso la atención lo resuelve dándole al decoder acceso a **todos** los estados del encoder, no solo al final.

Apuntes de clase — detalles del decoder

¿Por qué hay que agregar un `<sos>` al principio de la entrada del decoder? Porque en el primer paso el decoder todavía no generó ninguna palabra, y necesita **algo** como entrada para arrancar el primer paso de la LSTMCell (no hay "palabra anterior" real todavía). `<sos>` es un token especial reservado exactamente para eso: marcar el inicio y darle una entrada válida al primer paso. Del mismo modo, `<eos>` es el token especial que le indica al decoder **cuándo parar** al generar (en entrenamiento aparece al final de `tgt` ; en inferencia, generar es se detiene apenas el modelo lo produce).

La función `traducir()` del Colab: parte de un tensor de logits de forma **(B, L, V)** (por ejemplo (64, 9, 11.000): 64 oraciones, 9 posiciones generadas, ~11.000 palabras de vocabulario) y lo convierte en **oraciones de texto**: aplica `argmax` sobre la última dimensión (V) para quedarse con el índice más probable en cada posición → tensor de índices (B, L) → recorre cada fila y usa el vocabulario para pasar de índices a palabras, cortando en el primer `<eos>` que encuentra. Es el mismo patrón que "apilar" de la traducción voraz: de tensor de números a texto legible.

No es lo mismo traducir ES→EN que EN→ES: la última capa (Linear final) necesita una fila por palabra del **idioma de salida**, así que la cantidad de elementos de esa capa cambia según la dirección. Traducir hacia el idioma con vocabulario más chico da un modelo con menos parámetros en esa capa (en el notebook,

español→inglés: V_en=11.286 filas en la salida; el vocabulario español, 18.308, solo agranda la tabla de embeddings del encoder, que es más barata por fila que la capa de salida).

Split de datos: cuidado con las traducciones múltiples

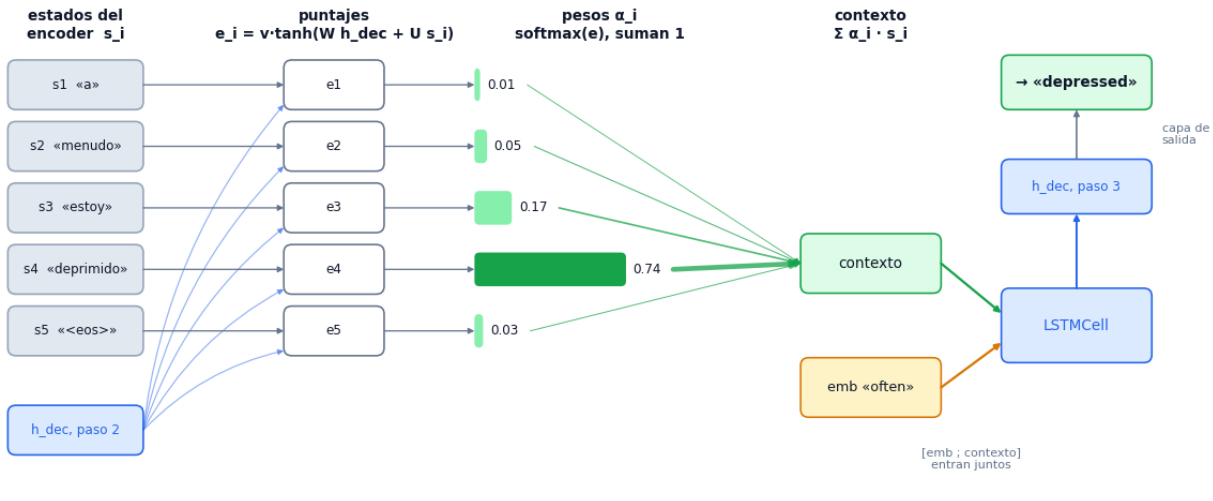
En el corpus español→inglés, una misma oración fuente puede aparecer con **varias traducciones** distintas (hasta 16 en el notebook). Si los pares se reparten al azar entre entrenamiento y validación, una oración fuente puede quedar en **los dos lados** con distinta traducción: el modelo "valida" sobre oraciones que ya vio, aunque con otra referencia. Esto infla BLEU (27,0 con split ingenuo vs. 25,6 con split correcto, que separa por oración fuente completa entre los conjuntos).

Clase 4 — Atención

Los estados que ya calculaba el encoder

La LSTM del encoder siempre devolvió dos cosas: `salidas` (B, L, oculta) —un estado por **posición**— y (h, c) —el estado **final**—. La Clase 3 (sin atención) tiraba `salidas` y usaba solo el estado final. `salidas[:, última]` es exactamente igual a `h_final` (diferencia 0,0): son 256 números por oración, contra 2.304 si se aprovechan todos los estados (256 × 9 posiciones, por ejemplo).

Atención de Bahdanau: el mecanismo paso a paso



Bahdanau: puntaje e_i = v·tanh(W h_dec + U s_i) → softmax(α_i) → contexto = Σ α_i·s_i

Paso 1: calcular un puntaje por cada estado del encoder, comparándolo con el estado del decoder: e_i = v · tanh(W·h_dec + U·s_i)

Paso 2: pasar los puntajes por una softmax → pesos α_i, positivos, que suman 1 (con −∞ en las posiciones de relleno)

Paso 3: calcular el contexto como la suma ponderada de los estados del encoder: c = Σ α_i·s_i

Paso 4: concatenar el contexto con el embedding de la palabra anterior y dar un paso de la `LSTMCell`

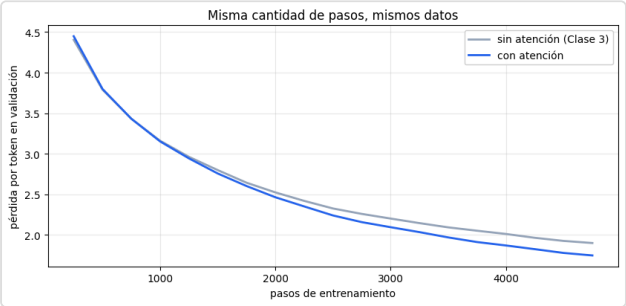
Paso 5: pasar el h nuevo por la capa de salida para obtener los logits de la palabra siguiente

En el ejemplo del notebook, al producir "depressed" en "a menudo estoy deprimido", el peso α sobre "deprimido" es **0,74** y el resto se reparte en migajas. Sin entrenar, los pesos son casi parejos (máximo ≈ 1/L); recién al entrenar se afilan sobre la palabra que corresponde.

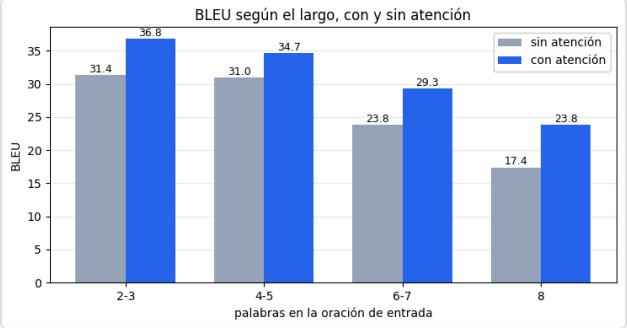
Por qué `LSTMCell` y no `LSTM`

> El traductor sin atención hace el decoder entero con una llamada a `nn.LSTM` sobre la secuencia completa (le entra toda de una vez). El traductor con atención usa `nn.LSTMCell` con un **bucle paso a paso**, porque la entrada de cada paso incluye el **contexto**, y ese contexto depende del estado del decoder del **paso anterior** (h_dec de t−1): no se puede armar la secuencia entera de antemano, porque el contexto del paso t todavía no existe hasta que el decoder ya dio ese paso. Es la misma celda matemática (diferencia numérica ~10^{-6}); lo que cambia es **quién escribe el bucle**. Costo: el forward tarda ~1,4× más y el modelo pasa de 7.434.262 a 7.827.734 parámetros.

Resultados con atención



Curvas de validación: se separan después de ~1.000 pasos



BLEU por largo: +5 puntos aprox. en todos los largos

Global: BLEU 26,2 (sin atención) → 31,2 (con atención). El notebook aclara que **no** se puede afirmar que la ganancia crezca sistemáticamente con el largo de la oración.

Mapas de alineación (de dónde sale la correspondencia entre palabras)

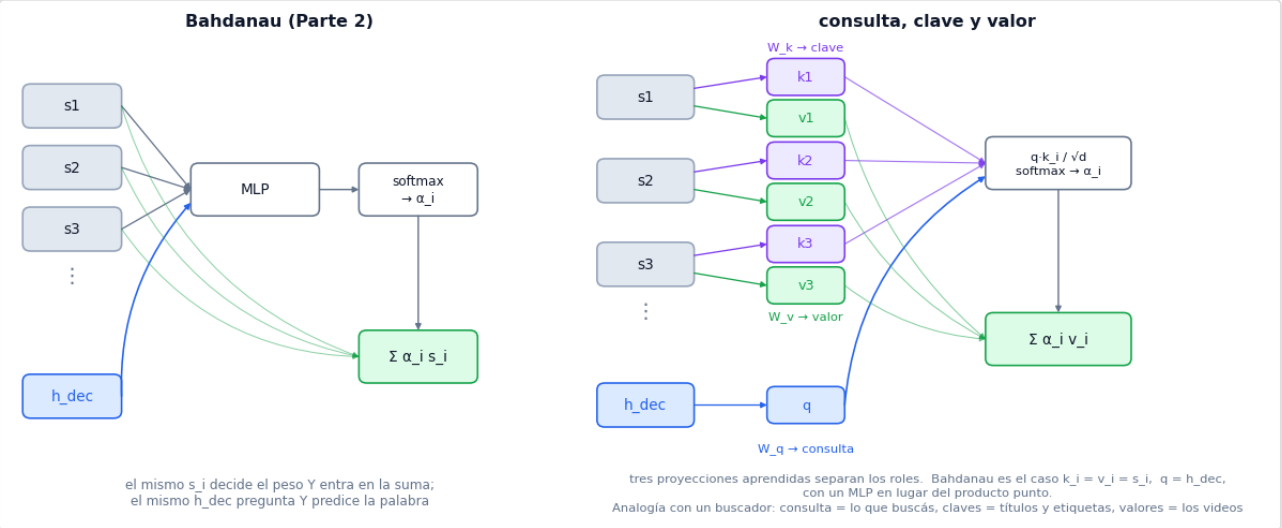


Filas = palabras generadas, columnas = palabras de entrada; cuanto más oscuro, más peso

Interpretación: los pesos se concentran sobre algo parecido a una **diagonal** (cada palabra generada mira sobre todo la palabra correspondiente de la entrada), y el bloque final se corre respecto de esa diagonal justo donde el inglés ordena distinto que el español (reordenamiento, ej. "ordenador nuevo" → "new computer"). El `<eos>` de salida suele mirar las últimas palabras de la entrada.

¿De dónde salió esa correspondencia? Nadie la definió: los pesos de atención se aprenden **junto con el resto del modelo**, como efecto secundario de aprender a traducir — es el título del paper de Bahdanau, "jointly learning to align and translate". No es un artefacto de la visualización (aunque cada fila suma 1 por construcción de la softmax, *dónde* se concentra esa masa sí es información real aprendida), ni viene de invertir la oración de entrada (eso se hace en la Clase 3 sin atención, por otro motivo), ni de un diccionario bilingüe externo.

Consulta, clave y valor (QKV): la formulación general



Bahdanau (caso particular) vs. QKV (formulación general con tres proyecciones separadas)

$$\text{puntaje} = q \cdot k_i / \sqrt{d} \;\rightarrow\; \text{softmax} \rightarrow \alpha_i \;\rightarrow\; \text{contexto} = \Sigma\, \alpha_i \cdot v_i$$

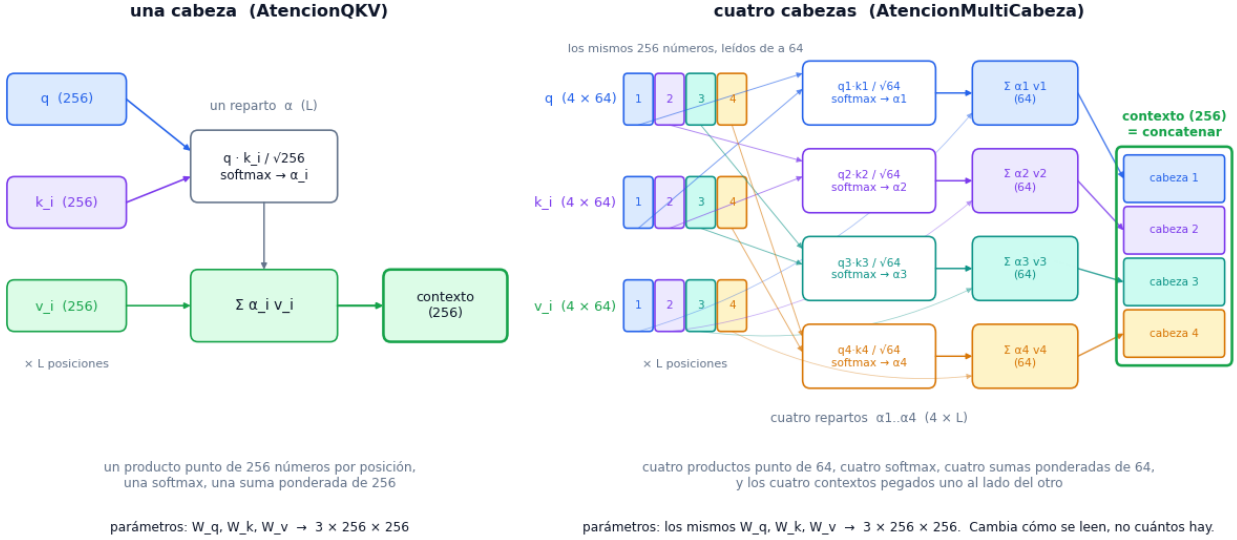
La **consulta (query)** q sale del decoder (proyección de h_dec con W_q). La **clave (key)** k_i decide el **peso** que recibe cada posición (proyección de cada s_i con W_k). El **valor (value)** v_i es lo que efectivamente se **suma** en el contexto (proyección de cada s_i con W_v). El puntaje ya no pasa por un

MLP con tanh como en Bahdanau, sino que es directamente un **producto punto escalado**: q·k_i/√d.

Bahdanau es el caso particular de QKV donde k_i = v_i = s_i (clave y valor son el mismo estado del encoder, sin proyección separada) y q = h_dec directo, con un MLP en vez de producto punto. La formulación QKV general, con **tres proyecciones aprendidas distintas**, separa los roles: el mismo vector puede servir para "predecir el token", "servir de clave para que otros lo encuentren" y "aportar contenido" de forma independiente, lo que da más flexibilidad al modelo para especializarse en cada rol — es el mecanismo que usan BERT, GPT y los transformers en general.

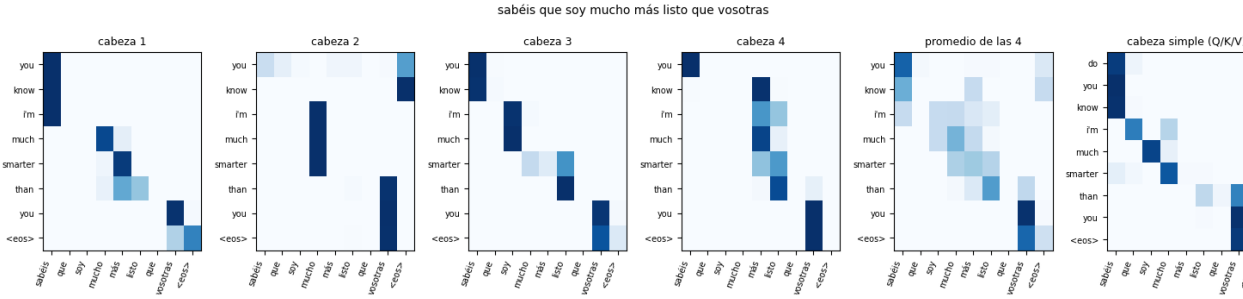
Analogía útil: consulta = lo que se busca, claves = títulos/etiquetas contra los que se compara, valores = el contenido que finalmente se devuelve (como un buscador).
Parámetros: Bahdanau (W, U, v) → 131.328. QKV (W_q, W_k, W_v) → 196.608. A pesar de tener más parámetros, QKV **no le gana** en BLEU a Bahdanau en este experimento (31,2 vs. 31,3 — prácticamente empatan). Más parámetros ahí no significa mejor traducción.

Atención multi-cabeza



Una cabeza de 256 vs. cuatro cabezas de 64: mismos parámetros, distinta forma de leerlos

Los mismos vectores de 256 dimensiones se **leen de a 64** (4 grupos). Cada cabeza calcula su propio reparto de pesos α y su propio contexto **de forma independiente**, y los cuatro contextos resultantes se **concatenan** al final (no se promedian). **No agrega parámetros**: siguen siendo los mismos W_q, W_k, W_v de 256×256 (196.608 en total); lo único que cambia es cómo se agrupan y leen esos números, no cuántos hay.



Mapas de alineación por cabeza: cada una se especializa en un patrón distinto

> Las cabezas 1 y 3 siguen la diagonal esperada (cada palabra mira a su correspondiente). La cabeza 2 mira sistemáticamente la palabra **anterior**, y la cabeza 4 una posición **más adelante**. Las cabezas se reparten distintos patrones de atención en vez de repetir todas lo mismo — es lo que justifica tener varias en vez de una sola. En este experimento puntual, sin embargo, BLEU no mejora al pasar de una cabeza a cuatro: el beneficio de multi-cabeza (distintos patrones especializados) no siempre se traduce en más BLEU a esta escala chica.

---

38. Los dos tipos de "pesos" en atención — nota de clase

> El profesor marcó este punto en clase porque en atención la palabra "pesos" se usa para **dos cosas distintas**, y el examen puede preguntar por cualquiera de las dos (o por la diferencia entre ambas).

1. Los pesos de atención (α_i) — "cuánto mira" cada posición

Son los números que salen de la **softmax** en cada paso del decoder y **suman 1**. **No son parámetros entrenables**: son una **salida** del cálculo, distinta en cada paso y en cada oración (no se guardan ni se reutilizan de un ejemplo a otro).

$$e_i = v \cdot \tanh(W\, h_{dec} + U\, s_i) \;\rightarrow\; \alpha_i = \mathrm{softmax}(e_i) \;\rightarrow\; c = \Sigma\, \alpha_i \cdot s_i$$

Esto es justo lo que se ve en los mapas de calor de alineación: cada fila (un paso del decoder) es una distribución de pesos α que suma 1 sobre las columnas (palabras de entrada). **Por qué "son mejores" / qué interpretación dan:** revelan qué parte de la oración fuente el modelo consideró relevante para generar cada palabra de salida — es una ventana a la correspondencia entre idiomas que el modelo **aprendió solo**, sin que nadie se la enseñara explícitamente. Cuanto más nítida y coherente con la intuición lingüística sea esa diagonal (con sus corrimientos en los reordenamientos), mejor señal de que el modelo está usando el contexto de forma sensata y no memorizando.

2. Los pesos entrenables (parámetros) de la capa de atención

Acá sí son **parámetros de verdad**: matrices que se ajustan con `backward()` como cualquier `nn.Linear`, y se guardan en el `state_dict` del modelo.

	Bahdanau	QKV (una cabeza)	Multi-cabeza (4×64)
Matrices	W, U, v (MLP)	W_q, W_k, W_v (producto punto)	Las mismas W_q, W_k, W_v
Parámetros	131.328	196.608	**196.608 (sin cambio)**
BLEU obtenido	31,3	31,2	no mejora respecto de 1 cabeza

Resumen para memorizar

	Pesos de atención (α)	Pesos entrenables (parámetros)
Qué son	Salida de una softmax, suman 1	Matrices que se ajustan con gradiente
Cuándo se fijan	En cada paso, en cada oración	Una vez, durante el entrenamiento
Multi-cabeza	Un α distinto por cabeza	Los mismos 196.608, sin agregar nada

> Formato esperado de pregunta sobre este tema: leer código y decir cuántos parámetros tiene una capa de atención, o emparejar conceptos (Bahdanau vs. QKV vs. multi-cabeza) según qué cambia y qué no cambia entre ellos.

Hoja de repaso rápido — fórmulas y números

Fórmulas

Concepto	Fórmula
Capa lineal vectorizada	$Z = X @ W.T + b$
Parámetros de nn.Linear(in, out)	in×out + out
Parámetros de una RNN (in, oculta h)	h·in + h + h·h + h = h(in+h+2)
Softmax	$\text{softmax}(z)_i = \exp(z_i) / \Sigma_j \exp(z_j)$
Entropía cruzada (un ejemplo)	−ln(p_correcta)
Pérdida inicial esperada (N clases)	ln(N)
L1 (MAE)	promedio(|pred − real|)
L2 (MSE)	promedio((pred − real)²)
Paso de descenso	$p \leftarrow p - lr \cdot p.grad$
RNN a mano	$h_t = \tanh(x_t W_{ih}^T + b_{ih} + h_{t-1} W_{hh}^T + b_{hh})$
Skip-gram, puntaje	$s(c,x) = v_c \cdot u_x$
Perplejidad	exp(entropía cruzada media); modelo uniforme = V
BLEU	brevedad × media_geométrica($p_1..p_4$)
Atención Bahdanau	$e_i=v\cdot\tanh(Wh_{dec}+Us_i) \rightarrow \alpha=\text{softmax}(e) \rightarrow c=\Sigma\alpha_i s_i$
Atención QKV	$\text{puntaje} = q\cdot k_i/\sqrt{d} \rightarrow \text{softmax} \rightarrow c=\Sigma\alpha_i v_i$
% de relleno en un tensor (B,L)	(L − largo_promedio) / L

Formas de tensores más preguntadas

Punto del pipeline	Forma
Lote de índices, truncado y rellenado	(B, L) — dtype int64
Máscara de relleno	(B, L) — dtype bool, True = token real
Después de nn.Embedding(V, E)	(B, L, E)
Después del promedio con máscara	(B, E)
Logits de un clasificador de N clases	(B, N)
salidas de una LSTM	(B, L, oculta) — un vector por posición
h, c de una LSTM (num_layers capas)	(num_layers, B, oculta)

Logits del decoder con teacher forcing	(B, L_en−1, V_en)
Logits de un modelo de lenguaje	(B, L, V)

Números de referencia (para no confundir de memoria)

Dato	Valor
Vocabulario de reseñas (freq_min=1)	~59.045 tokens; con freq_min=50: ~5.882
Percentiles de largo de reseñas	mediana 25-26 · p90=59 · p99=123-127
Parámetros del clasificador de reseñas	1.485.253 (99,4% en la tabla de embeddings)
ln(5) — pérdida inicial, 5 clases	1,609
Vocabulario de cuentos (modelado de lenguaje)	5.003 tokens
Perplejidad LSTM vs. mejor n-grama	73,1 (LSTM) vs. 100,8 (trigrama interpolado)
Gradiente RNN simple vs. LSTM (posición 0)	8 órdenes de magnitud de diferencia
BLEU sin atención → con atención	26,2 → 31,2
Parámetros atención Bahdanau vs. QKV	131.328 vs. 196.608 (BLEU casi igual)
Parámetros multi-cabeza (4×64) vs. 1 cabeza (256)	Iguales: 196.608

Errores silenciosos a los que el banco vuelve una y otra vez

- Aplicar `softmax` en el `forward` y además usar `nn.CrossEntropyLoss` → entrena, pero mucho peor, sin ningún error.
- No llamar `zero_grad()` antes de `backward()` → los gradientes se acumulan entre lotes.
- No usar `pack_padded_sequence` en un clasificador recurrente → la misma reseña da estados finales distintos según el lote.
- No barajar (`shuffle=True`) un corpus ordenado por clase → validación puede quedar con una sola clase.
- Olvidar `modelo.eval()` con dropout activo → métricas ruidosas sin error visible.
- Usar `salidas[:, -1]` en vez de `h[-1]` con relleno → puede ser una posición de puro `<pad>`.
- Normalizar índices de vocabulario como si fueran cantidades → son casilleros, no números.